

ULTIMATE TEACH YOURSELF BOOKLET

Computer Science XI

Learn every chapter on your own — explanations, worked examples, memory tricks, practice questions and full answer keys.

Bilingual support: English + اردو

کمپیوٹر سائنس — گیارہویں جماعت: خود سیکھیں، خود پرکھیں

02 Computational Thinking & Algorithm

03 Programming Fundamentals

05 Application and Impacts of Computing

Contents

Chapter 2: Computational Thinking & Algorithm

- 2.1 Computational Thinking
- 2.2 Algorithm
- 2.2.2 Algorithm vs Pseudocode
- 2.3.1 Decomposition
- 2.3.2 Pattern Recognition
- 2.3.3 Abstraction
- 2.4 Sorting and Searching Algorithms
- 2.4.1 Sorting Algorithms: Bubble Sort
- 2.4.1 Sorting Algorithms: Selection Sort
- 2.4.2 Searching Algorithms: Linear Search
- 2.4.2 Searching Algorithms: Binary Search
- 2.5 Evaluation Criteria of Algorithm
- 2.6 How to Decide Which Algorithm to Use
- Chapter 2 Review

Chapter 3: Programming Fundamentals

- 3.1 Programming
- 3.1.1 Programming Language Types
- 3.2 Career Growth with Python
- 3.3 Integrated Development Environment (IDE)
- 3.3.1 Visual Studio Code (VS Code)
- 3.4 Fundamentals of Python Programming
- 3.4.1 Variables
- 3.4.1 Data Types in Python
- 3.4.2 Input and Output Handling
- 3.4.3 Operators and Operands: Arithmetic and Assignment
- 3.4.3 Relational, Logical and Membership Operators
- 3.4.3 Bitwise Operators
- 3.5 Control Structures
- 3.5.1 Sequence
- 3.5.2 Selection Statements: if and if-else
- 3.5.2 Selection Statements: if-elif-else and Nested if

Chapter 5: Application and Impacts of Computing

- 5.1 Introduction to Computing Impacts
- 5.1.1 Artificial Intelligence (AI)
- 5.1.2 Internet of Things (IoT) & Components
- 5.1.3 Data Analytics & Components
- Comparison: IoT vs AI vs Data Analytics
- 5.2 Uses of IoT in Different Areas
- 5.2.1 Scope of AI in Education in Pakistan
- 5.2.2 Career Connection in IoT and AI
- 5.3 Information Sources (Primary, Secondary, Tertiary)
- 5.4 Impacts of Computing in Various Fields
- 5.5 Assistive Technologies
- 5.5.1 Importance of Assistive Technologies
- 5.5.2 Career Connection in Assistive Tech
- Chapter 5 Review

How to use this booklet

This booklet is written so that you can learn every chapter **on your own**, without a teacher sitting next to you. Each topic explains the idea in simple English, shows a worked example, gives you a memory trick, warns you about common mistakes, explains it again in Urdu, and ends with quick questions so you can check that you really understood.

یہ کتابچہ اس طرح لکھا گیا ہے کہ آپ استاد کے بغیر خود ہر باب سیکھ سکیں۔ ہر موضوع پہلے آسان انگریزی میں سمجھایا گیا ہے، پھر حل شدہ مثال، یاد رکھنے کا طریقہ، عام غلطیاں، اردو میں وضاحت اور آخر میں خود کو پرکھنے کے سوالات دیے گئے ہیں۔

What the coloured boxes mean

LEARN IT

The main explanation. Read it slowly and make sure every sentence makes sense.

WORKED EXAMPLE

A solved example, step by step. Cover the solution and try it yourself first.

REMEMBER IT

A short trick, analogy or mnemonic that helps you recall the idea in the exam.

COMMON MISTAKE

Mistakes students often make. Read these before every test.

اردو میں سمجھیں
اسی موضوع کی اردو میں آسان وضاحت۔

EXAM TIP

How Golden topics are asked in board exams and what examiners look for.

CHECK YOURSELF

Two or three quick questions. Answer them before looking at the answers underneath.

GOLDEN TOPIC

★ **Golden** marks a high-yield topic that appears often in examinations. Give these topics extra time.

The 5-step self-study method

1. **Preview (5 minutes).** Read the chapter opener and the "What you will learn" list. Turn each objective into a question you want answered.
2. **Learn one topic at a time.** Read the explanation, then work through the example with a pen and paper. Do not just read the solution — reproduce it.
3. **Check yourself straight away.** Answer the "Check yourself" questions from memory. If you get one wrong, re-read that topic before moving on.
4. **Practise at the end of the chapter.** Do the multiple-choice, short and long questions in exam conditions (no notes), then mark your work using the answer key.
5. **Review and repeat.** Tick the "I can..." checklist. Revisit anything you could not tick after 1 day, 3 days and 7 days. Spaced repetition makes knowledge stick.

Tips for board exam answers

- **Define, explain, example.** A strong short answer gives a clear definition, one or two lines of explanation and a real-life example.

- **Use tables for "differentiate" questions.** Compare at least four points side by side.
- **Draw and label.** Logic gates, diagrams, charts and data structures earn marks when drawn neatly and labelled.
- **Show every step.** In K-maps, sorting traces, trace tables, statistics and code output questions, examiners give marks for the working, not only the final answer.
- **Write correct code.** Indent Python carefully, use meaningful variable names, and write the expected output under your program.

Computational Thinking & Algorithm

This chapter teaches you how to think like a computer scientist: break a problem into parts, spot patterns, ignore what does not matter, and write clear step-by-step algorithms. You will then learn four famous algorithms — Bubble Sort, Selection Sort, Linear Search and Binary Search — and how to judge and choose between them.

اس باب میں آپ سیکھیں گے کہ کسی مسئلے کو کمپیوٹر سائنس دان کی طرح کیسے سوچ کر حل کیا جاتا ہے: مسئلے کو چھوٹے حصوں میں توڑنا، پیٹرن پہچاننا، غیر ضروری باتوں کو نظر انداز کرنا اور مرحلہ وار الگورتھم لکھنا۔ اس کے بعد آپ ترتیب دینے (Sorting) اور تلاش کرنے (Searching) کے چار اہم الگورتھم سیکھیں گے۔

What you will learn

- Define computational thinking and name its four parts: decomposition, pattern recognition, abstraction and algorithms.
- Define an algorithm and state five reasons why algorithms are important.
- Compare an algorithm with pseudocode on at least five points.
- Apply decomposition, pattern recognition and abstraction to real-life problems.
- Dry-run Bubble Sort and Selection Sort pass by pass on a given list, and write them in pseudocode and Python.
- Trace Linear Search step by step and Binary Search using a low / mid / high table.
- List the advantages and disadvantages of each sorting and searching algorithm.
- State the best-case and worst-case time complexity of the four algorithms.
- Evaluate an algorithm for correctness (using a trace table), efficiency and clarity.
- Choose the right algorithm for a situation and justify your choice.

2.1 Computational Thinking

LEARN IT

Computational Thinking (CT) is a systematic approach to solving problems. "Systematic" means you follow a fixed, organised method instead of guessing. CT involves four activities:

- **Decomposition** — breaking a big problem down into smaller parts.
- **Pattern recognition** — identifying things that repeat or follow a rule.
- **Abstraction** — hiding (ignoring) details that do not matter.
- **Algorithms** — developing clear step-by-step instructions to solve the problem.

CT is not "thinking like a computer". It is thinking in a way that lets *you* describe a solution so clearly that a computer (or another person) could carry it out.

Problem → Decomposition → Pattern recognition → Abstraction → Algorithm → Program

You already use computational thinking in daily life. When you plan your exam preparation, you split the syllabus into chapters (decomposition), notice that every chapter needs the same routine of reading,

making notes and practising (pattern recognition), ignore things that are not in the syllabus (abstraction), and make a day-by-day timetable (algorithm).

LEARN IT

Did you know? Pseudocode is a human-readable representation of an algorithm using plain English and structured programming keywords. It is the bridge between human logic and machine code. You will study it in detail in section 2.2.2.

REMEMBER IT

Remember the four parts of CT with the phrase "**Don't Pick Any Apple**": **D**ecomposition, **P**attern recognition, **A**bstraction, **A**lgorithm.

COMMON MISTAKE

Students sometimes think computational thinking means "writing code". It does not. CT happens *before* coding. You can do all four steps with pen and paper, without a computer.

اردو میں سمجھیں
کمپیوٹیشنل تھنکنگ (Computational Thinking) مسائل حل کرنے کا ایک منظم طریقہ ہے۔ اس کے چار حصے ہیں: مسئلے کو چھوٹے حصوں میں توڑنا (Decomposition)، دہرائی جانے والی باتوں کو پہچاننا (Pattern Recognition)، غیر ضروری تفصیلات کو چھپانا (Abstraction) اور مرحلہ وار حل لکھنا (Algorithm)۔ یہ سوچنے کا طریقہ ہے، کوڈ لکھنا نہیں۔

CHECK YOURSELF

1. Name the four parts of computational thinking.
2. Is computational thinking done before or after writing code?
3. What is pseudocode called a "bridge" between?

Answers: 1. Decomposition, pattern recognition, abstraction, algorithms. 2. Before coding (it is planning the solution). 3. Between human logic and machine code.

2.2 Algorithm ★ Golden

LEARN IT

An **algorithm** is a step-by-step logical procedure used to solve a problem. It is **conceptual** in nature (it is an idea, a plan) and completely **independent of any programming language**. The same algorithm can be written later in Python, C++ or Java.

A good algorithm has:

- **Input** — the data it starts with.
- **Clear steps** — each step has only one meaning.
- **A definite end** — it must finish after a limited number of steps.
- **Output** — the result it produces.

WORKED EXAMPLE

Algorithm to find the larger of two numbers

Step 1: Start

Step 2: Read A and B

Step 3: If $A > B$, print "A is larger"

Step 4: Else print "B is larger or equal"

Step 5: End

Notice: there is no Python or C++ here. Anyone — even a non-programmer — can follow these steps. That is what "language-independent" means.

2.2.1 Importance of Algorithms

Reason	What it means for you
Structured approach	Algorithms provide a structured approach to problem-solving, making complex tasks manageable.
Optimisation	They optimise time and memory usage, which is crucial for large-scale applications (for example, searching millions of records).
Reusability	They can be reused for similar problems, saving time and effort. A sorting algorithm for marks can also sort prices.
Accuracy and consistency	They ensure accuracy and consistency, reducing human error. The same steps always give the same result.
Blueprint	They act as a blueprint (a plan) before actual code implementation, just like an architect's map before building a house.

REMEMBER IT

Remember the five reasons as **S-O-R-A-B**: **S**tructured, **O**ptimised, **R**eusable, **A**ccurate, **B**lueprint.

COMMON MISTAKE

Do not write "an algorithm is a program". A program is the algorithm *written in a programming language* so that a computer can run it. The algorithm itself is only the logic.

اردو میں سمجھیں
الگورتھم (Algorithm) کسی مسئلے کو حل کرنے کا مرحلہ وار منطقی طریقہ ہے۔ یہ ایک تصور ہے اور کسی بھی پروگرامنگ زبان پر منحصر نہیں۔ الگورتھم کام کو منظم بناتے ہیں، وقت اور میموری بچاتے ہیں، دوبارہ استعمال ہو سکتے ہیں، غلطیاں کم کرتے ہیں اور کوڈ لکھنے سے پہلے نقشے کا کام دیتے ہیں۔

EXAM TIP

Very common short question: "Define algorithm and write its importance." Write the definition word for word — *a step-by-step logical procedure used to solve a problem; conceptual and independent of any programming language* — then list at least four of the five importance points. If asked to "write an algorithm", always begin with **Step 1: Start** and finish with **End**, and number every step.

CHECK YOURSELF

1. Why is an algorithm called "language-independent"?
2. Give two reasons why algorithms are important.
3. Write a 5-step algorithm to find the sum of two numbers.

Answers: 1. Because it describes only the logic; it is not tied to Python, C++ or any other language. 2. Any two: structured approach, optimise time/memory, reusable, accurate and consistent, blueprint before coding. 3. Step 1: Start; Step 2: Read A and B; Step 3: SUM = A + B; Step 4: Print SUM; Step 5: End.

2.2.2 Algorithm vs Pseudocode ★ Golden

LEARN IT

Both algorithms and pseudocode are used in problem-solving, but they serve different purposes in the design phase. The **algorithm** says *what* must be done. The **pseudocode** shows *how* it could be implemented, using programming-like keywords such as IF, FOR and WHILE, but it still cannot be run on a computer.

Point	Algorithm	Pseudocode
Definition	A step-by-step logical procedure to solve a problem.	A human-readable representation of an algorithm using plain English & keywords.
Nature	Conceptual and language-independent.	Semi-formal and structured, but not executable code.
Purpose	To define what needs to be done logically.	To present how it can be implemented in a readable format.
Form	Usually described in plain steps, bullets or numbered points.	Written using structured constructs (IF, FOR, WHILE) like programming.
Focus	Focuses on the logic and steps of the problem.	Focuses on representing steps clearly before coding.
Audience	For anyone trying to understand the logic (including non-programmers).	Mainly for programmers or students preparing for coding.
Format rules	No fixed format; can be informal.	Follows standard pseudocode conventions for clarity.

WORKED EXAMPLE

The same problem written both ways: print whether a number is even or odd.

Algorithm

Step 1: Start
Step 2: Read a number N
Step 3: Divide N by 2 and find the remainder
Step 4: If the remainder is 0, print "Even", otherwise print "Odd"
Step 5: End

Pseudocode

```
START
INPUT N
SET R TO N MOD 2
IF R = 0 THEN
    PRINT "Even"
ELSE
    PRINT "Odd"
END IF
END
```

The equivalent Python program (real, executable code):

```
n = 7
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

OUTPUT
Odd

Problem → Algorithm (what) → Pseudocode (how) → Program (code) → Output

REMEMBER IT

Teacher's analogy: an **algorithm** is like a recipe ("Bake a cake at 350 degrees"). **Pseudocode** is like a structured kitchen manual using standard verbs (SET temperature TO 350; WHILE cake is not baked, WAIT). The program is the oven actually doing it.

COMMON MISTAKE

Pseudocode is **not** executable. If you type pseudocode into Python, it will give errors. Also, do not mix exact Python syntax (colons, brackets) into pseudocode answers unless asked — use keywords like IF ... THEN ... END IF.

اردو میں سمجھیں
الگورتھم بتاتا ہے کہ کیا کرنا ہے، جبکہ سوڈو کوڈ (Pseudocode) بتاتا ہے کہ اسے کیسے کیا جا سکتا ہے۔ الگورتھم سادہ مرحلوں میں لکھا جاتا ہے اور ہر کوئی اسے سمجھ سکتا ہے۔ سوڈو کوڈ میں IF، FOR اور WHILE جیسے الفاظ استعمال ہوتے ہیں اور یہ زیادہ تر پروگرامز کے لیے ہوتا ہے، لیکن اسے کمپیوٹر پر چلایا نہیں جا سکتا۔

EXAM TIP

"Differentiate between algorithm and pseudocode" is a favourite 4–5 mark question. Draw a two-column table and give at least **five** points (definition, nature, purpose, form, audience). Adding the recipe vs kitchen-manual analogy and one small example of each earns full marks.

CHECK YOURSELF

1. Which one — algorithm or pseudocode — uses keywords like FOR and WHILE?
2. Which one is meant for non-programmers too?
3. Can pseudocode be executed by a computer?

Answers: 1. Pseudocode. 2. Algorithm. 3. No; it is semi-formal and not executable.

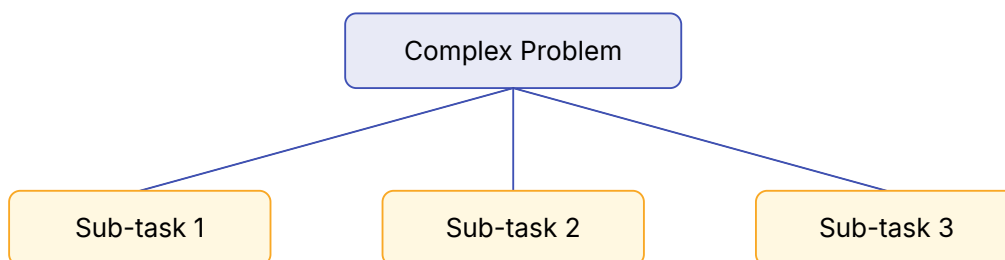
2.3.1 Decomposition ★ Golden

2.3 Algorithmic Strategies for Problem Solving. An **algorithmic strategy** is a general method that helps you design an algorithm. The three strategies in your syllabus are decomposition, pattern recognition and abstraction.

LEARN IT

Decomposition is a problem-solving technique in which a complex task or system is broken down into smaller, manageable parts. By dividing a big problem into simpler steps, it becomes easier to understand, develop and solve.

Each small part is called a **sub-task** (or sub-problem). You solve each sub-task separately, then join the solutions to solve the whole problem.



Decomposition: one complex problem is split into smaller sub-tasks.

WORKED EXAMPLE

Real-life example: Learning a new language

"Learn English" (or any language) is too big to do in one go. Decompose it:

Sub-task	Description
Learn vocabulary	Learn the words for people, actions and things (e.g., "I", "eat", "apple").
Grammar rules	Learn the sentence structure: Subject → Verb → Object.
Learn tenses	Present, past, future (e.g., "I eat" vs. "I ate").
Sentence building	Combine words using the rules to create valid sentences: "I ate an apple."

Vocabulary → Grammar rules → Tenses → Sentence building

WORKED EXAMPLE

Computing example: a "Student Result" program

Problem: read the marks of a student in 5 subjects and print the total, percentage and grade.

- Sub-task 1: Input marks of 5 subjects.
- Sub-task 2: Calculate total = sum of the 5 marks.
- Sub-task 3: Calculate percentage = $\text{total} \div 500 \times 100$.
- Sub-task 4: Decide the grade from the percentage.
- Sub-task 5: Print the result card.

Each sub-task is now so small that you can write it in two or three lines of code.

Why is decomposition useful?

- **Reduces complexity:** you focus on one piece at a time.
- **Helps in debugging:** you can find exactly where the mistake is (e.g., in the language example, is the error in the verb tense or the word order?).
- **Promotes reusability:** once you learn or build a small part, you can apply it elsewhere (the "calculate percentage" part can be reused in an attendance program).

REMEMBER IT

Think of eating a big *roti*: you cannot eat it in one bite, so you break it into small pieces.

Decomposition = "break the roti".

COMMON MISTAKE

Do not break a problem into random pieces. Each sub-task must be a meaningful, clear job, and all sub-tasks together must solve the *whole* problem. If one part is missing (e.g., "print result"), the solution is incomplete.

اردو میں سمجھیں
ڈیکمپوزیشن (Decomposition) کا مطلب ہے کسی بڑے اور پیچیدہ مسئلے کو چھوٹے اور آسان حصوں میں تقسیم کرنا۔ مثلاً نئی زبان سیکھنے کو الفاظ، گرامر، زمانے (Tenses) اور جملہ سازی میں تقسیم کیا جاسکتا ہے۔ اس سے پیچیدگی کم ہوتی ہے، غلطی ڈھونڈنا آسان ہوتا ہے اور حصے دوبارہ استعمال ہو سکتے ہیں۔

EXAM TIP

Expect "Define decomposition with a real-life example" or "Explain the benefits of decomposition". Give the definition, draw the small tree (Complex Problem → Sub-task 1, 2, 3), show the language-learning table, and list the three benefits: reduces complexity, helps debugging, promotes reusability.

CHECK YOURSELF

1. What is each smaller part of a decomposed problem called?
2. Decompose "Making a school timetable" into three sub-tasks.
3. State the three benefits of decomposition.

Answers: 1. A sub-task (sub-problem). 2. Example: list subjects and teachers; decide periods per subject per week; place periods in days so no teacher has two classes at once. 3. Reduces complexity, helps in debugging, promotes reusability.

2.3.2 Pattern Recognition

LEARN IT

Pattern recognition is the process of identifying regularities, repetitions or rules in data. It helps in reducing the overall complexity of an algorithm and forms the foundation of concepts such as **loops**.

How it works — ask yourself three questions:

- What is repeating?
- What changes in a predictable way?
- Can I generalise this (write one rule that works for every case)?

WORKED EXAMPLE

Example: Triangle of stars

OUTPUT

```
*  
**  
***  
****  
*****
```

Step 1 — What repeats? Every row prints stars.

Step 2 — What changes predictably? Each row adds 1 star to the previous row's count.

Row	Stars
1	1
2	2
3	3
...	...
N	N

Step 3 — Generalise: Row N = N stars. Recognising this pattern lets us write one generic loop instead of 5 separate print statements.

Without the pattern (5 statements)

```
print("*")  
print("**")  
print("***")  
print("****")  
print("*****")
```

With the pattern (one loop)

```
n = 5  
for row in range(1, n + 1):  
    print("*" * row)
```

Both programs print exactly the same triangle. But if you want 100 rows, the loop version needs only one change (`n = 100`), while the first version needs 100 lines.

REMEMBER IT

Whenever you see "the same thing again and again, with a small change each time", think **loop**. Pattern recognition is the habit of spotting that.

COMMON MISTAKE

Do not generalise from too few cases. Check your rule on several rows (or items) before trusting it. "Row N = N stars" is correct here; a different triangle (1, 3, 5, 7 stars) follows the rule "Row N = 2N - 1 stars".

پیمٹرن ریکگنیشن (Pattern Recognition) کا مطلب ہے ڈیٹا میں دہرائی جانے والی چیزوں یا اصولوں کو پہچاننا۔ ستاروں کی تکون میں ہر قطار میں پچھلی قطار سے ایک ستارہ زیادہ ہے، یعنی قطار N میں N ستارے۔ یہ پیمٹرن پہچان کر ہم پانچ الگ پرنٹ کے بجائے ایک لوپ (Loop) لکھ سکتے ہیں۔

CHECK YOURSELF

1. Which programming concept is based on pattern recognition?
2. How many stars will row 8 of the triangle have?
3. Find the rule: 2, 4, 6, 8, ...

Answers: 1. Loops. 2. 8 stars (Row N = N stars). 3. Term N = 2 × N (each term adds 2).

2.3.3 Abstraction

LEARN IT

Abstraction means hiding unnecessary features and focusing only on the essential ones needed to solve a problem. It reduces complexity by ignoring what is not important (the **noise**).

When you apply abstraction you sort details into three groups:

- **Necessary steps** — the core logic that never changes.
- **Changes based on the situation** — these become **variables** (inputs) in the algorithm.
- **Abstraction items (noise)** — details you can ignore completely.

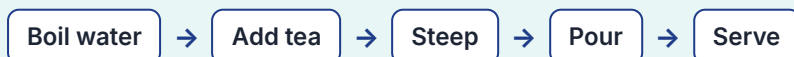
WORKED EXAMPLE

Example: applying abstraction to "Making Tea"

To make an algorithm for tea, we separate the necessary steps from the situational changes and the noise.

1. Necessary steps (core logic)	2. Changes based on situation (variables)	3. Abstraction items (ignore / noise)
1. Boil water 2. Add tea leaves/bag 3. Steep for a few minutes 4. Pour into cup 5. Serve	Ingredients (black, green, milk) Additions (sugar, lemon) Tools (stove vs kettle)	Brand of kettle Cup colour or design Where tea is made (kitchen, office)

The core abstracted logic, independent of cup colour or kettle brand:



This one algorithm now works for green tea in an office kettle and for milk tea on a kitchen stove. The middle column simply becomes inputs: `tea_type`, `addition`, `tool`.

Computers use abstraction everywhere. When you call `print("Hello")` in Python, you do not think about how the screen lights up each pixel — those details are hidden from you. A map of Karachi shows roads

and landmarks but hides every tree and car: it is an abstraction of the city.

REMEMBER IT

Abstraction = "**Keep the skeleton, drop the decoration.**" Ask: "If I remove this detail, does the solution still work?" If yes, it is noise.

COMMON MISTAKE

Do not confuse abstraction with decomposition. Decomposition *splits* a problem into parts. Abstraction *removes* unimportant details. Also, do not throw away details that really matter — "boil water" can never be ignored, but kettle brand can.

اردو میں سمجھیں

ابسٹرکشن (Abstraction) کا مطلب ہے غیر ضروری تفصیلات کو چھپا کر صرف ضروری باتوں پر توجہ دینا۔ چائے بنانے کی مثال میں پانی ابالنا، پتی ڈالنا، دم دینا، کپ میں ڈالنا اور پیش کرنا ضروری مراحل ہیں؛ چائے کی قسم یا چینی متغیرات (Variables) ہیں؛ جبکہ کیتلی کا برانڈ یا کپ کا رنگ فالتو باتیں (Noise) ہیں جنہیں نظر انداز کیا جاتا ہے۔

CHECK YOURSELF

1. What is "noise" in abstraction?
2. In the tea example, is "sugar or lemon" a necessary step, a variable, or noise?
3. How is abstraction different from decomposition?

Answers: 1. Unnecessary details that do not affect the solution. 2. A variable (it changes with the situation). 3. Decomposition breaks a problem into parts; abstraction hides details that are not important.

2.4 Sorting and Searching Algorithms

LEARN IT

Sorting and searching are both fundamental techniques of computer science. They work on **lists of the same data type** (for example, a list of all integers or a list of all names).

Sorting	Searching
Arranging a collection of items in an order. The order may be ascending (0–9, A–Z) or descending (9–0, Z–A). Used to organise data.	Locating a specific element within a list to find its position or check its existence (is it there or not?).
Examples: Bubble Sort, Selection Sort	Examples: Linear Search, Binary Search
Real life: merit list arranged from highest to lowest marks.	Real life: finding your roll number in the merit list.

Notation you will see

- **DATA** — the list (array). `DATA[1]` is the 1st element in the textbook algorithms (counting starts at 1).
- In **pseudocode** and **Python** counting starts at **0**: `DATA[0]` is the 1st element. This is called an **index**.
- **N** — the number of elements in the list.
- **PTR** (pointer) — a variable that holds the current position.

- **temp** — a temporary variable used to **swap** (exchange) two values.
- **Pass** — one complete run of the inner loop over the list.

WORKED EXAMPLE

How a swap works with temp. Suppose $A = 8$ and $B = 4$. To exchange them:

$\text{temp} = A \text{ (temp} = 8\text{)} \rightarrow A = B \text{ (} A = 4\text{)} \rightarrow B = \text{temp (} B = 8\text{)}$

Without `temp`, writing $A = B$ first would destroy the 8. In Python you may also write $a, b = b, a$.

COMMON MISTAKE

Position 1 in the textbook algorithm is index 0 in Python. In exams, if a question uses `DATA[1]`, count from 1; if it uses Python, count from 0. State clearly which one you use.

اردو میں سمجھیں

سارٹنگ (Sorting) کا مطلب ہے ڈیٹا کو صعودی (چھوٹے سے بڑے) یا نزولی (بڑے سے چھوٹے) ترتیب میں رکھنا۔ سرچنگ (Searching) کا مطلب ہے فہرست میں کسی خاص چیز کو تلاش کرنا تاکہ اس کی جگہ یا موجودگی معلوم ہو سکے۔ دونوں ایک ہی قسم کے ڈیٹا کی فہرست پر کام کرتے ہیں۔

CHECK YOURSELF

1. Arrange C, A, D, B in descending order.
2. What is the purpose of the variable `temp`?

Answers: 1. D, C, B, A. 2. It temporarily holds one value so that two values can be swapped without losing either.

2.4.1 Sorting Algorithms: Bubble Sort ★ Golden

LEARN IT

Bubble Sort is an elementary sorting algorithm that works by repeatedly comparing **adjacent** (side-by-side) elements and swapping them if they are in the wrong order.

In each pass, the largest remaining value "bubbles up" to the end of the list, like an air bubble rising to the top of water. So after pass 1 the largest value is in the last place, after pass 2 the second largest is in second-last place, and so on. A list of N items needs **$N - 1$ passes**, and pass K makes **$N - K$ comparisons**.

Compare `DATA[j]` and `DATA[j+1]` → Left > Right? → Yes: swap → Move one step right →
End of pass: largest is at the end → Repeat $N-1$ passes

Algorithm (textbook)

Pseudocode

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N-1
Step 5: Set PTR := 1
Step 6: Repeat while PTR ≤ N-K:
    a: If DATA[PTR] > DATA[PTR+1]:
        1. Set temp = DATA[PTR]
        2. Set DATA[PTR] = DATA[PTR+1]
        3. Set DATA[PTR+1] = temp
    b: Set PTR := PTR + 1
Step 7: End

```

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = 0 to N-2-outer
        IF DATA[inner] > DATA[inner+1]
            Temp = DATA[inner]
            DATA[inner] = DATA[inner+1]
            DATA[inner+1] = Temp
        End If
    End For
End For
End

```

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order. The pair in brackets [] is being compared. Positions are counted from 1.

Pass 1 (K = 1, 4 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[8, 4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
2nd vs 3rd	4, [8, 1], 9, 3	8 > 1 Yes	Swap	4, 1, 8, 9, 3
3rd vs 4th	4, 1, [8, 9], 3	8 > 9 No	No change	4, 1, 8, 9, 3
4th vs 5th	4, 1, 8, [9, 3]	9 > 3 Yes	Swap	4, 1, 8, 3, 9

9 is now at its correct (final) place.

Pass 2 (K = 2, 3 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[4, 1], 8, 3, 9	4 > 1 Yes	Swap	1, 4, 8, 3, 9
2nd vs 3rd	1, [4, 8], 3, 9	4 > 8 No	No change	1, 4, 8, 3, 9
3rd vs 4th	1, 4, [8, 3], 9	8 > 3 Yes	Swap	1, 4, 3, 8, 9

Pass 3 (K = 3, 2 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 4], 3, 8, 9	1 > 4 No	No change	1, 4, 3, 8, 9
2nd vs 3rd	1, [4, 3], 8, 9	4 > 3 Yes	Swap	1, 3, 4, 8, 9

Pass 4 (K = 4, 1 comparison)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 3], 4, 8, 9	1 > 3 No	No change	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 + 3 + 2 + 1 = **10 comparisons** and **6 swaps**.

Pass	List after pass	Fixed at the end
Start	8, 4, 1, 9, 3	—
1	4, 1, 8, 3, 9	9
2	1, 4, 3, 8, 9	8, 9
3	1, 3, 4, 8, 9	4, 8, 9
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Bubble Sort in Python (follows the pseudocode exactly; Python indexes start at 0):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [4, 1, 8, 3, 9]
After pass 2 : [1, 4, 3, 8, 9]
After pass 3 : [1, 3, 4, 8, 9]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

`range(n - 1 - outer)` gives `inner = 0 ... N-2-outer`, exactly as in the pseudocode. Notice the list was already sorted after pass 3, but the plain algorithm still does pass 4.

Advantages and disadvantages

Advantages	Disadvantages
Easiest algorithm to visualise and explain.	Uses nested loops, so the time grows very fast as the list grows (about N^2 comparisons).
Does not need extra memory (in-place sorting).	Performs a massive number of swaps.
Stable (equal items stay in their original order).	Useless for real-world "Big Data".

REMEMBER IT

Bubble = neighbours. Bubble Sort only ever compares *neighbours* (j and $j+1$). Each pass pushes the biggest bubble to the end. For **descending** order, just change `>` to `<` in the comparison.

COMMON MISTAKE

The slides say the time grows "exponentially". Strictly, Bubble Sort's time grows **quadratically**: for N items it makes $N(N-1)/2$ comparisons, which is $O(N^2)$. Doubling the list makes it about 4 times slower — bad, but not exponential. Another common mistake: students forget that pass K does only $N - K$ comparisons; the sorted tail is never compared again.

بل سارٹ (Bubble Sort) میں ساتھ ساتھ موجود دو عناصر کا موازنہ کیا جاتا ہے اور اگر وہ غلط ترتیب میں ہوں تو ان کی جگہ بدل دی جاتی ہے۔ ہر پاس (Pass) کے بعد سب سے بڑا عدد آخر میں پہنچ جاتا ہے۔ N عناصر کے لیے $N-1$ پاس درکار ہوتے ہیں۔ یہ سمجھنے میں آسان اور مستحکم (Stable) ہے لیکن بڑے ڈیٹا کے لیے بہت سست ہے۔

EXAM TIP

The most common question: "Sort the list ... using Bubble Sort. Show each pass." Draw one table per pass with columns *Pair*, *Compare*, *Action*, *List after*. Mark the element fixed at the end after every pass. Also be ready to write the algorithm (with PTR and K) or the pseudocode (with outer/inner), and to list two advantages and two disadvantages.

CHECK YOURSELF

1. How many passes and comparisons does Bubble Sort need for 6 elements?
2. Show the list 5, 1, 4 after pass 1 of Bubble Sort.
3. Sort 2, 9, 4, 7 in descending order with Bubble Sort: give the list after each pass.

Answers: 1. 5 passes; $5 + 4 + 3 + 2 + 1 = 15$ comparisons. 2. (5,1) swap $\rightarrow$ 1, 5, 4; (5,4) swap $\rightarrow$ 1, 4, 5. 3. Pass 1: 9, 4, 7, 2; Pass 2: 9, 7, 4, 2; Pass 3: 9, 7, 4, 2.

2.4.1 Sorting Algorithms: Selection Sort ★ Golden

LEARN IT

Selection Sort is a simple sorting algorithm that works by repeatedly **selecting the smallest element** from the unsorted portion of the list and moving it to its correct position at the front.

Think of it as filling seats from the front: first find who belongs in position 1, then position 2, and so on. In the textbook version, the element at position K is compared with every element after it; whenever a smaller one is found, the two are swapped. So at the end of pass K , the K -th smallest value sits in position K .



Algorithm (textbook)

Pseudocode

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N
Step 5: Set PTR := K+1
Step 6: Repeat while PTR ≤ N:
    a: If DATA[K] > DATA[PTR]:
        1. Set temp = DATA[K]
        2. Set DATA[K] = DATA[PTR]
        3. Set DATA[PTR] = temp
    b: Set PTR := PTR + 1
Step 7: End

```

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = outer+1 to N-1
        IF DATA[outer] > DATA[inner]
            Temp = DATA[outer]
            DATA[outer] = DATA[inner]
            DATA[inner] = Temp
        End If
    End For
End For
End

```

REMEMBER IT

Teacher's tip — spot the difference: in Selection Sort the inner loop starts at `outer + 1` and compares with the *fixed* position `DATA[outer]`. In Bubble Sort the inner loop always starts at `0` and compares *neighbours* `DATA[inner]` and `DATA[inner+1]`. Bubble pushes the largest to the **end**; Selection brings the smallest to the **front**.

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order (textbook method). The two positions in brackets are being compared.

Pass 1 — find the smallest for Position 1

Compare	List	Pos1 > PosX?	Action	List after
Pos1 vs Pos2	[8], [4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
Pos1 vs Pos3	[4], 8, [1], 9, 3	4 > 1 Yes	Swap	1, 8, 4, 9, 3
Pos1 vs Pos4	[1], 8, 4, [9], 3	1 > 9 No	No change	1, 8, 4, 9, 3
Pos1 vs Pos5	[1], 8, 4, 9, [3]	1 > 3 No	No change	1, 8, 4, 9, 3

1 is now permanently in Position 1.

Pass 2 — find the smallest for Position 2

Compare	List	Pos2 > PosX?	Action	List after
Pos2 vs Pos3	1, [8], [4], 9, 3	8 > 4 Yes	Swap	1, 4, 8, 9, 3
Pos2 vs Pos4	1, [4], 8, [9], 3	4 > 9 No	No change	1, 4, 8, 9, 3
Pos2 vs Pos5	1, [4], 8, 9, [3]	4 > 3 Yes	Swap	1, 3, 8, 9, 4

Pass 3 — find the smallest for Position 3

Compare	List	Pos3 > PosX?	Action	List after
Pos3 vs Pos4	1, 3, [8], [9], 4	8 > 9 No	No change	1, 3, 8, 9, 4
Pos3 vs Pos5	1, 3, [8], 9, [4]	8 > 4 Yes	Swap	1, 3, 4, 9, 8

Pass 4 — find the smallest for Position 4

Compare	List	Pos4 > PosX?	Action	List after
Pos4 vs Pos5	1, 3, 4, [9], [8]	9 > 8 Yes	Swap	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 + 3 + 2 + 1 = **10 comparisons, 6 swaps** in this textbook method.

Pass	List after pass	Fixed at the front
Start	8, 4, 1, 9, 3	—
1	1, 8, 4, 9, 3	1
2	1, 3, 8, 9, 4	1, 3
3	1, 3, 4, 9, 8	1, 3, 4
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Selection Sort in Python (follows the textbook pseudocode):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(outer + 1, n):
        if data[outer] > data[inner]:
            temp = data[outer]
            data[outer] = data[inner]
            data[inner] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [1, 8, 4, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

WORKED EXAMPLE

The "classic" Selection Sort (one swap per pass). Many books first *find the position of the smallest* value in the unsorted part, and swap only once at the end of each pass. The result is the same; only the swaps are fewer.

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    smallest = outer
    for inner in range(outer + 1, n):
        if data[inner] < data[smallest]:
            smallest = inner
    data[outer], data[smallest] = data[smallest], data[outer]
    print("After pass", outer + 1, ":", data)
```

OUTPUT

```
After pass 1 : [1, 4, 8, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
```

Here only 4 swaps are made (at most $N - 1$). This is why Selection Sort is said to perform fewer swaps than Bubble Sort. In exams, use the textbook method unless the question asks otherwise.

Advantages and disadvantages

Advantages	Disadvantages
Very easy to code.	Time grows very fast with list size (nested loops, about N^2 comparisons).
Performs fewer swaps overall compared to Bubble Sort.	Must finish every single loop — it is "blind" to an already sorted list.
Does not need extra memory.	Not stable (it might change the relative positions of equal items).

WORKED EXAMPLE

Why "not stable"? Take two equal marks, 5_a (Ali) and 5_b (Bilal), in the list $5_a, 5_b, 2$. Pass 1 compares 5_a with 5_b (no swap), then 5_a with $2 \rightarrow \text{swap} \rightarrow 2, 5_b, 5_a$. Now Bilal is before Ali, although Ali came first originally. Bubble Sort never does this, because it swaps only neighbours that are strictly out of order.

COMMON MISTAKE

In the textbook method, the comparison is always with the **fixed position** $\text{DATA}[K]$ (Pos1 in pass 1, Pos2 in pass 2), not with the neighbour. Students often mix this up with Bubble Sort and compare neighbours. Also, after a swap, the *new* value at Pos K is used for the next comparison.

اردو میں سمجھیں

سلیکشن سارٹ (Selection Sort) میں فہرست کے غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔ ہر پاس میں ایک مقررہ جگہ (Position) کا موازنہ اس کے بعد والے تمام عناصر سے ہوتا ہے۔ اس کا کوڈ آسان ہے اور بیل سارٹ سے کم تبادلوں (Swaps) ہوتے ہیں، لیکن یہ مستحکم (Stable) نہیں اور ترتیب شدہ فہرست پر بھی پورے لوپ چلاتا ہے۔

EXAM TIP

Typical questions: "Sort ... using Selection Sort, showing all steps", "Differentiate between Bubble Sort and Selection Sort" and "Why is Selection Sort not stable?" In the difference question mention: what is compared (neighbours vs fixed position), where the inner loop starts (0 vs outer+1), where items settle (end vs front), stability, and number of swaps.

CHECK YOURSELF

1. In Selection Sort, where does the inner loop start?
2. Using the textbook method, show 7, 2, 5 after pass 1.
3. Which of the two sorts is stable?

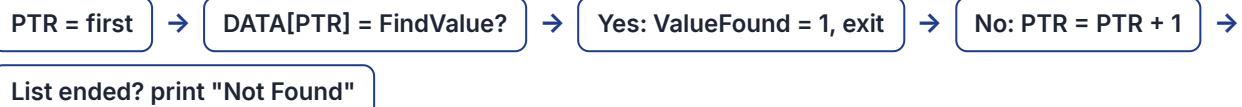
Answers: 1. At outer + 1 (PTR = K + 1). 2. Pos1 vs Pos2: $7 > 2 \rightarrow \text{swap} \rightarrow 2, 7, 5$; Pos1 vs Pos3: $2 > 5$ no $\rightarrow 2, 7, 5$. 3. Bubble Sort.

2.4.2 Searching Algorithms: Linear Search

LEARN IT

Linear Search, also known as **sequential search**, works by examining each element in the list one by one, from the start, until the target is found (or the list ends). It works on both **sorted and unsorted** data.

A **flag** variable (here `ValueFound`) starts at 0 and becomes 1 when the item is found. After the loop, the flag tells us whether to print the item or "Not Found".



Algorithm (textbook)

```
Step 1: Start
Step 2: DATA := {8,3,6,1,7,2,4,5}
Step 3: N := 8
Step 4: FindValue := 7
Step 5: ValueFound := 0
Step 6: Repeat for PTR = 1 to N
Step 7:   If DATA[PTR] = FindValue:
           a. ValueFound = 1
           b. Goto Step 8
Step 8:   If ValueFound = 1:
           Print DATA[PTR]
         Else:
           Print "Not Found"
Step 9: End
```

Pseudocode

```
Start
DATA = {8, 3, 6, 1, 7, 2, 4, 5}
N = 8
FindValue = 7
ValueFound = 0
For PTR = 0 to N-1
    If DATA[PTR] = FindValue then
        ValueFound = 1
        Exit Loop
    End If
End For
If ValueFound = 1 then
    Print DATA[PTR]
Else
    Print "Data Not Found"
End If
End
```

WORKED EXAMPLE

Worked example. Data list: 8, 3, 6, 1, 7, 2, 4, 5. Searching item: 7.

Step	Element checked	Condition (== 7)	Result
1	8	8 == 7 ?	Not found
2	3	3 == 7 ?	Not found
3	6	6 == 7 ?	Not found
4	1	1 == 7 ?	Not found
5	7	7 == 7 ?	Found! (Exit loop)

7 is found at position 5 (index 4) after 5 comparisons. The remaining items 2, 4, 5 are never checked.

If the item is missing — e.g., search for 9 — all 8 elements are checked, the flag stays 0, and "Data Not Found" is printed. This is the **worst case**: N comparisons.

WORKED EXAMPLE

Linear Search in Python

```
data = [8, 3, 6, 1, 7, 2, 4, 5]
find_value = 7
value_found = 0
for ptr in range(len(data)):
    print("Step", ptr + 1, ": checking", data[ptr])
    if data[ptr] == find_value:
        value_found = 1
        break
if value_found == 1:
    print("Found", data[ptr], "at position", ptr + 1)
else:
    print("Data Not Found")
```

OUTPUT

```
Step 1 : checking 8
Step 2 : checking 3
Step 3 : checking 6
Step 4 : checking 1
Step 5 : checking 7
Found 7 at position 5
```

`break` is Python's "Exit Loop".

Advantages and disadvantages

Advantages	Disadvantages
Does not care whether data is scrambled or sorted.	If the target is at the end of 1 million items, it does 1 million checks.
Extremely simple code (one loop, one if statement).	Time grows linearly (double the list = double the time).
Faster for very small lists (no setup time, such as sorting).	Inefficient if you need to search the same list thousands of times.
Does not require any extra space.	

REMEMBER IT

Linear Search is like looking for your friend in a queue by checking every face from the front. Best case: 1 check (first person). Worst case: N checks (last person or not there).

COMMON MISTAKE

Do not print "Not Found" inside the loop after every mismatch — that would print it many times. Decide only **after** the loop ends, using the flag. And remember to **exit the loop** once the item is found, or you waste comparisons.

لینیئر سرچ (Linear Search) کو ترتیب وار تلاش (Sequential Search) بھی کہتے ہیں۔ اس میں فہرست کے ہر عنصر کو شروع سے ایک ایک کر کے چیک کیا جاتا ہے جب تک مطلوبہ چیز نہ مل جائے۔ یہ ترتیب شدہ اور غیر ترتیب شدہ دونوں قسم کے ڈیٹا پر کام کرتی ہے اور بہت سادہ ہے، لیکن بڑی فہرست میں بہت وقت لیتی ہے۔

CHECK YOURSELF

1. What is another name for Linear Search?
2. In 8, 3, 6, 1, 7, 2, 4, 5, how many comparisons are needed to find 2?
3. What is the purpose of the variable ValueFound?

Answers: 1. Sequential search. 2. 6 comparisons (2 is in position 6). 3. It is a flag: 0 means not found yet, 1 means found; it decides what to print after the loop.

2.4.2 Searching Algorithms: Binary Search ★ Golden

LEARN IT

Binary Search is a highly efficient search algorithm that works by repeatedly dividing the search interval in half. **Crucial limitation: it only works on a SORTED list.**

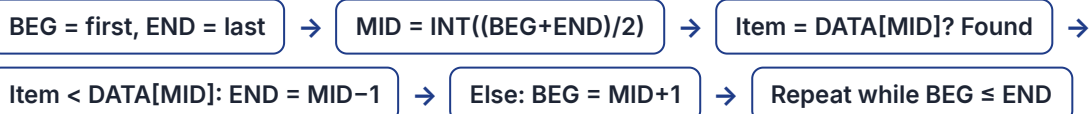
Three variables mark the part of the list still being searched:

- **BEG** (low) — first position of the current part.
- **END** (high) — last position of the current part.
- **MID** = $\text{INT}((\text{BEG} + \text{END}) / 2)$ — the middle position. INT means "drop the decimal part".

At each step compare the item with $\text{DATA}[\text{MID}]$:

- Equal $\rightarrow$ found, stop.
- Item $< \text{DATA}[\text{MID}] \rightarrow$ the item can only be in the **left** half: set $\text{END} = \text{MID} - 1$.
- Item $> \text{DATA}[\text{MID}] \rightarrow$ the item can only be in the **right** half: set $\text{BEG} = \text{MID} + 1$.

If **BEG** becomes greater than **END**, the part is empty and the item is **not in the list**.



Algorithm (textbook summary)

Pseudocode

```

Step 1: Start
Step 2: DATA := {1,2,3,4,5,6,7,8,9}
        (Sorted)
Step 3: BEG = 1, END = 9
Step 4: MID = INT((BEG+END)/2)
Step 5: Item = 7
Step 6: LOC = 0
Step 7: Repeat while BEG ≤ END AND
        DATA[MID] ≠ Item
Step 8:   If Item < DATA[MID]:
            END = MID - 1
        Else:
            BEG = MID + 1
Step 9:   MID = INT((BEG+END)/2)
Step 10: [End loop]
Step 11: If DATA[MID] = Item:
            LOC = MID
        Else:
            LOC = 0
Step 12: Print result based on LOC
Step 13: End

```

```

Start
DATA = {1,2,3,4,5,6,7,8,9}
BEG = 1, END = 9
MID = INT((BEG+END)/2)
Item = 7
LOC = NULL
While BEG ≤ END AND DATA[MID] ≠ Item
    IF Item < DATA[MID] then
        END = MID - 1
    Else
        BEG = MID + 1
    End If
    MID = INT((BEG+END)/2)
End While
If DATA[MID] = Item then
    LOC = MID
Else
    LOC = NULL
End If
If LOC ≠ NULL then
    Print DATA[MID]
Else
    Print "Not Found"
End If
End

```

COMMON MISTAKE

The slide's pseudocode writes the loop condition as $BEG \leq END \text{ OR } DATA[MID] \neq Item$. That is a slip: with OR the loop would never stop once the item is found while $BEG \leq END$. The textbook algorithm (Step 7) correctly uses **AND** — keep going only while the part is not empty *and* the item is not yet found. Likewise, the slide sets $LOC = NULL$ but then tests " $LOC \geq 0$ "; the check should simply be " LOC is not NULL" (or, with $LOC = 0$ meaning not found, " $LOC > 0$ "). The pseudocode above shows the corrected form.

WORKED EXAMPLE

Worked example 1 (textbook algorithm). DATA = 1, 2, 3, 4, 5, 6, 7, 8, 9; Item = 7. Positions 1 to 9.

Step	BEG (low)	END (high)	MID	DATA[MID]	Compare	Action
1	1	9	$INT(10/2) = 5$	5	$7 > 5$	BEG = 6 (go right)
2	6	9	$INT(15/2) = 7$	7	$7 = 7$	Found, LOC = 7

Only **2 comparisons**. Linear Search would need 7.

WORKED EXAMPLE

Worked example 2 (slide example). Sorted list: 1, 2, 3, 4, 5, 6, 7, 8. Searching item: 3.

Step	BEG (low)	END (high)	MID	DATA[MID]	Compare	Action
1	1	8	$\text{INT}(9/2) = 4$	4	$3 < 4$	END = 3 (go left)
2	1	3	$\text{INT}(4/2) = 2$	2	$3 > 2$	BEG = 3 (go right)
3	3	3	$\text{INT}(6/2) = 3$	3	$3 = 3$	FOUND at position 3

Picture the list shrinking (the highlighted cell is MID):

Step	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	1	2	3					
3			3					

The slide describes the same steps using the size of each sub-list: Step 1 $\text{MID} = 8/2 = 4$ (value 4), $3 < 4$, go left; Step 2 sub-list [1, 2, 3], middle = $3/2 = 1.5 \rightarrow$ 2nd item (value 2), $3 > 2$, go right; Step 3 sub-list [3], middle = 1st item (value 3), FOUND. Both ways land on the same values $4 \rightarrow 2 \rightarrow 3$.

WORKED EXAMPLE

Binary Search in Python (index starts at 0, so positions 1–8 become indexes 0–7):

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
item = 3
beg = 0
end = len(data) - 1
loc = -1
step = 0
while beg ≤ end:
    mid = (beg + end) // 2
    step = step + 1
    print("Step", step, ": beg =", beg, "mid =", mid, "end =", end, "value =", data[mid])
    if data[mid] == item:
        loc = mid
        break
    elif item < data[mid]:
        end = mid - 1
    else:
        beg = mid + 1
if loc ≥ 0:
    print("Found", item, "at index", loc)
else:
    print("Not Found")
```

OUTPUT

```
Step 1 : beg = 0 mid = 3 end = 7 value = 4
Step 2 : beg = 0 mid = 1 end = 2 value = 2
Step 3 : beg = 2 mid = 2 end = 2 value = 3
Found 3 at index 2
```

// is integer division, the same as INT(). Index 2 is position 3. Here `loc = -1` means "not found", so the test `loc ≥ 0` is correct.

Why is it so fast?

Every comparison throws away half of the remaining list. $1,000,000 \rightarrow 500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$ takes about 20 halvings, because $2^{20} = 1,048,576$. So Binary Search finds an item among 1 million elements in at most about **20 steps**, while Linear Search may need 1,000,000.

Advantages and disadvantages

Advantages	Disadvantages
Extremely fast for "Big Data": finds an item in 1M elements in just ~20 steps.	The list must be sorted first; it fails on scrambled data.
Massive time savings for static lists that are searched repeatedly.	Requires random access to elements (works on arrays, not easily on linked lists).
	More complex code, prone to logic errors (off-by-one).

REMEMBER IT

Binary Search is how you find a word in a dictionary: open the middle, decide "before or after?", and throw away half the book each time. **"Sorted first, then split."**

COMMON MISTAKE

Common errors: (1) using Binary Search on an unsorted list — the answer will be wrong; (2) setting $END = MID$ instead of $MID - 1$ (or $BEG = MID$ instead of $MID + 1$), which can cause an infinite loop — this is the "off-by-one" error; (3) forgetting INT , so MID becomes a decimal like 4.5.

اردو میں سمجھیں

بانٹری سرچ (Binary Search) ایک تیز رفتار طریقہ ہے جو ہر قدم پر تلاش کے حصے کو آدھا کر دیتا ہے۔ یہ صرف ترتیب شدہ (Sorted) فہرست پر کام کرتا ہے۔ درمیانی عنصر (MID) سے موازنہ کیا جاتا ہے: اگر مطلوبہ چیز چھوٹی ہو تو بائیں آدھے حصے میں، اور بڑی ہو تو دائیں آدھے حصے میں تلاش جاری رہتی ہے۔ دس لاکھ عناصر میں بھی تقریباً بیس قدموں میں نتیجہ مل جاتا ہے۔

EXAM TIP

You will usually be given a sorted list and an item and asked to "search using Binary Search, showing all steps". Always draw a table with **BEG, END, MID, DATA[MID], comparison, action**. Show the INT calculation for MID . If the item is absent, continue until $BEG > END$ and write "Not Found". Also learn: "Why must the list be sorted?" and "Compare Linear and Binary Search".

CHECK YOURSELF

1. What is the crucial condition for Binary Search to work?
2. $BEG = 6$, $END = 9$. What is MID ?
3. In 1, 2, 3, 4, 5, 6, 7, 8 (positions 1–8), list the MID values visited when searching for 8.

Answers: 1. The list must be sorted. 2. $INT(15/2) = 7$. 3. $MID = 4$ (value 4, go right, $BEG = 5$) $\rightarrow MID = INT(13/2) = 6$ (value 6, go right, $BEG = 7$) $\rightarrow MID = INT(15/2) = 7$ (value 7, go right, $BEG = 8$) $\rightarrow MID = 8$ (value 8, found). Four comparisons.

2.5 Evaluation Criteria of Algorithm

LEARN IT

Many algorithms can solve the same problem. To judge which is better, we evaluate them on three criteria: **correctness**, **efficiency** and **clarity**.

2.5.1 Correctness

An algorithm is **correct** if:

- its output matches the expected outcomes;
- it handles **edge cases** gracefully (unusual inputs such as an empty list, 0, a negative number, or an item that is not present);
- it is guaranteed to finish — no infinite loops.

We test correctness with a **trace table** (dry run): we feed sample inputs and record every value by hand, then compare the result with what we expect.

WORKED EXAMPLE

Trace table example: Even/Odd checker. Rule: divide the number by 2; if the remainder is 0 it is Even, otherwise Odd.

Input	Operation (divide by 2, find remainder)	Remainder == 0?	Result
3	3 / 2 (Quotient = 1, Remainder = 1)	No	Odd
6	6 / 2 (Quotient = 3, Remainder = 0)	Yes	Even
15	15 / 2 (Quotient = 7, Remainder = 1)	No	Odd

All three results match what we expect, so the algorithm is correct for these inputs. The same check in Python:

```
for number in [3, 6, 15]:
    remainder = number % 2
    if remainder == 0:
        print(number, "remainder", remainder, "→ Even")
    else:
        print(number, "remainder", remainder, "→ Odd")
```

OUTPUT

```
3 remainder 1 → Odd
6 remainder 0 → Even
15 remainder 1 → Odd
```

A good tester would also try the edge case 0 (remainder 0 → Even, which is correct).

2.5.2 Efficiency

We measure algorithm efficiency through **time** (speed) and **space** (memory). The key question is: how fast does it run as the input size grows — does it **scale** well?

Computer scientists describe growth with **Big O notation**. It counts roughly how many steps are needed for N items:

- **O(1)** — constant: same time, whatever N is.
- **O(log N)** — logarithmic: grows very slowly (Binary Search).
- **O(N)** — linear: double N, double the time (Linear Search).
- **O(N²)** — quadratic: double N, four times the time (Bubble and Selection Sort).

WORKED EXAMPLE

How the worst-case number of comparisons grows

N (items)	Linear Search (N)	Binary Search (about $\log_2 N$)	Bubble/Selection Sort $N(N-1)/2$
10	10	4	45
100	100	7	4,950
1,000	1,000	10	499,500
1,000,000	1,000,000	20	499,999,500,000

This is why Binary Search is called excellent for Big Data and the simple sorts are called useless for it.

2.5.3 Clarity (Readability)

An algorithm must be easy to read, understand and modify. It should use **clear steps**, **meaningful variable names** and **good indentation**.

Unclear

```
a=[3,6,15]
for x in a:
    if x%2==0:print(x,"E")
    else:print(x,"O")
```

Clear

```
numbers = [3, 6, 15]
for number in numbers:
    if number % 2 == 0:
        print(number, "Even")
    else:
        print(number, "Odd")
```

Both work, but the second is far easier for another person (or you, a month later) to understand and change.

REMEMBER IT

Remember **C-E-C**: **C**orrect (right answer, always finishes), **E**fficient (fast and light), **C**lear (easy to read).

COMMON MISTAKE

Testing only "normal" inputs is not enough to prove correctness. Always include edge cases in your trace table, e.g., the first item, the last item, a missing item, or an empty list in a search.

اردو میں سمجھیں
کسی الگورتھم کو تین معیارات پر پرکھا جاتا ہے: درستگی (Correctness) یعنی درست نتیجہ دے، غیر معمولی صورتوں کو سنبھالے اور ہمیشہ ختم ہو؛ کارکردگی (Efficiency) یعنی کم وقت اور کم میموری استعمال کرے؛ اور وضاحت (Clarity) یعنی پڑھنے، سمجھنے اور تبدیل کرنے میں آسان ہو۔ درستگی جانچنے کے لیے ٹریس ٹیبل (Trace Table) بنایا جاتا ہے۔

CHECK YOURSELF

1. Name the three evaluation criteria of an algorithm.
2. Give one example of an edge case for a search algorithm.
3. Efficiency is measured in terms of which two resources?

Answers: 1. Correctness, efficiency, clarity. 2. The item is not in the list (or the list is empty, or the item is the last element). 3. Time (speed) and space (memory).

2.6 How to Decide Which Algorithm to Use

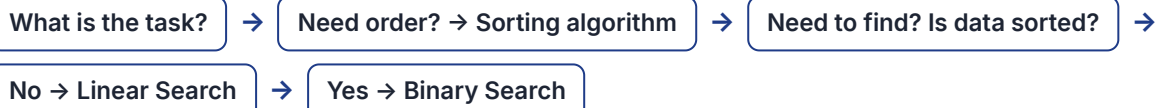
LEARN IT

Before choosing an algorithm, ask three questions:

- **What is the input size?** (10 items or 10 million?)
- **Is the data sorted?**
- **Do we need speed or memory efficiency?**

Simple decision guide:

- If searching on **unsorted** data → use **Linear Search**.
- If searching on **sorted** data → use **Binary Search**.
- If data needs **ordering** → use a **sorting** algorithm.



Comparison of the four algorithms

Algorithm	Type	Best case	Worst case	Extra memory	Needs sorted data?	Stable?
Bubble Sort	Sorting	$O(N^2)$ as written; $O(N)$ with an early-stop check	$O(N^2)$	None — $O(1)$	—	Yes
Selection Sort	Sorting	$O(N^2)$	$O(N^2)$	None — $O(1)$	—	No
Linear Search	Searching	$O(1)$ (first item)	$O(N)$ (last item / absent)	None — $O(1)$	No	—
Binary Search	Searching	$O(1)$ (item at first MID)	$O(\log N)$	None — $O(1)$	Yes	—

Point	Bubble Sort	Selection Sort
What is compared	Adjacent elements (j and j+1)	Fixed position K with every later element
Inner loop starts at	0	outer + 1
After each pass	Largest moves to the end	Smallest moves to the front
Swaps	Many	Fewer
Stable	Yes	No

Point	Linear Search	Binary Search
Method	Checks one by one	Splits the list in half each step
Data required	Sorted or unsorted	Sorted only
Worst case (1M items)	1,000,000 checks	About 20 checks
Code	Very simple	More complex, off-by-one risk
Best for	Small or unsorted lists, one-off searches	Large, sorted lists searched many times

WORKED EXAMPLE

Choosing in practice

- **A shopkeeper's list of 15 items, written in random order, searched once:** Linear Search — the list is unsorted and tiny; sorting first would waste time.
- **NADRA looking up a CNIC number among millions of sorted records, thousands of times a day:** Binary Search — data is sorted, huge, and searched repeatedly.
- **A teacher arranging 40 students' marks into a merit list:** a sorting algorithm; for 40 items even Bubble or Selection Sort is acceptable.
- **An unsorted list of 1 million records that will be searched thousands of times:** sort it once, then use Binary Search for every search.

REMEMBER IT

Two keywords decide most exam answers: "**sorted?**" and "**how big?**". Unsorted or small → Linear. Sorted and big → Binary.

COMMON MISTAKE

"Binary Search is always better" is wrong. On unsorted data it does not work at all, and for a single search on a small unsorted list, sorting first costs more than a simple Linear Search.

اردو میں سمجھیں
الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔ غیر ترتیب شدہ ڈیٹا میں تلاش کے لیے لینئر سرچ، ترتیب شدہ ڈیٹا میں تلاش کے لیے بائنری سرچ، اور ڈیٹا کو ترتیب دینے کے لیے سارٹنگ الگورتھم استعمال کریں۔

CHECK YOURSELF

1. Which search would you use on an unsorted list of phone contacts?
2. What is the worst-case time complexity of Binary Search?
3. Which sort has the same time complexity in its best and worst case?

Answers: 1. Linear Search. 2. $O(\log N)$. 3. Selection Sort ($O(N^2)$ in both).

Chapter 2 Review

Chapter summary

- **Computational thinking** is systematic problem solving using decomposition, pattern recognition, abstraction and algorithms.
- An **algorithm** is a step-by-step logical procedure to solve a problem; it is conceptual and language-independent.
- Algorithms give structure, optimise time and memory, are reusable, ensure accuracy and act as a blueprint for code.
- **Pseudocode** is a human-readable, structured representation of an algorithm using keywords like IF, FOR and WHILE; it is not executable.
- An algorithm defines *what* to do; pseudocode shows *how* to implement it.
- **Decomposition** breaks a big problem into smaller sub-tasks; it reduces complexity, helps debugging and promotes reusability.
- **Pattern recognition** finds repetitions and rules (Row N = N stars) and is the foundation of loops.
- **Abstraction** hides noise and keeps only essential steps; situational details become variables.
- Sorting arranges data (ascending or descending); searching locates an element's position or existence.
- **Bubble Sort** compares and swaps adjacent elements; the largest value moves to the end each pass; N – 1 passes; stable; $O(N^2)$.
- **Selection Sort** brings the smallest remaining value to the front each pass; inner loop starts at outer + 1; fewer swaps; not stable; $O(N^2)$.
- **Linear Search** checks items one by one; works on unsorted data; $O(N)$ worst case.
- **Binary Search** halves the search range using BEG, MID and END; needs sorted data; $O(\log N)$ — about 20 steps for 1 million items.
- Algorithms are evaluated on **correctness** (checked with trace tables and edge cases), **efficiency** (time and space) and **clarity**.
- Choose by asking: input size, sorted or not, speed or memory? Unsorted → Linear; sorted → Binary; needs order → sort.

Key terms

Term	Meaning	اردو
Computational thinking	A systematic approach to solving problems using decomposition, patterns, abstraction and algorithms.	کمپیوٹیشنل سوچ
Algorithm	A step-by-step logical procedure to solve a problem, independent of any language.	الگورتھم، مرحلہ وار طریقہ حل
Pseudocode	A human-readable, structured but non-executable representation of an algorithm.	سوڈو کوڈ، فرضی کوڈ
Decomposition	Breaking a complex problem into smaller, manageable parts.	تقسیم کاری
Sub-task	One small part of a decomposed problem.	ذیلی کام
Pattern recognition	Identifying regularities, repetitions or rules in data.	نمونے کی پہچان
Abstraction	Hiding unnecessary details and focusing on essentials.	تجربہ، غیر ضروری تفصیل چھپانا
Noise	Details that do not affect the solution and can be ignored.	فالتو تفصیلات
Sorting	Arranging items in ascending or descending order.	ترتیب دینا
Searching	Locating an element in a list to find its position or existence.	تلاش کرنا
Ascending order	Smallest to largest (0-9, A-Z).	صعودی ترتیب
Descending order	Largest to smallest (9-0, Z-A).	نزولی ترتیب
Bubble Sort	Sort that repeatedly compares and swaps adjacent elements.	بل سارٹ
Selection Sort	Sort that repeatedly selects the smallest element and puts it at the front.	سلیکشن سارٹ
Pass	One full run of the inner loop through the list.	ایک چکر
Swap	Exchanging the values of two positions, usually with a temp variable.	جگہ کا تبادلہ
In-place sorting	Sorting without needing extra memory for another list.	اضافی میموری کے بغیر ترتیب
Stable sort	A sort that keeps equal items in their original order.	مستحکم ترتیب
Linear (sequential) search	Checking each element one by one until the target is found.	ترتیب وار تلاش
Binary search	Search that halves a sorted list at each step.	دو حصی تلاش
BEG / MID / END	First, middle and last positions of the part being searched in Binary Search.	آغاز / درمیان / اختتام
Flag	A variable (like ValueFound) that records whether something has happened.	نشان دہی کرنے والا متغیر

Correctness	Giving expected output, handling edge cases and always finishing.	درستی
Efficiency	How well an algorithm uses time and memory as input grows.	کارکردگی
Clarity	How easy an algorithm is to read, understand and modify.	وضاحت
Trace table	A table recording values step by step during a dry run.	ٹریس ٹیبل، جانچ جدول
Edge case	An unusual or extreme input used to test an algorithm.	انتہائی صورت
Time complexity (Big O)	A measure of how the number of steps grows with input size, e.g. $O(N)$, $O(\log N)$.	وقتی پیچیدگی

Practice: multiple choice

- An algorithm is best defined as:
 - a programming language
 - a step-by-step logical procedure to solve a problem
 - a type of computer hardware
 - compiled machine code
- Pseudocode is:
 - executable machine code
 - a symbol used in flowcharts
 - a Python library
 - a human-readable representation of an algorithm using plain English and keywords
- Breaking a complex problem into smaller, manageable parts is called:
 - decomposition
 - abstraction
 - pattern recognition
 - compilation
- In the "making tea" example, ignoring the brand of the kettle is an example of:
 - decomposition
 - sorting
 - abstraction
 - searching
- Pattern recognition forms the foundation of:
 - variables
 - loops
 - comments
 - input devices
- Bubble Sort works by comparing:
 - the first and last elements
 - every element with the middle element
 - adjacent elements
 - random pairs of elements
- After the first pass of Bubble Sort (ascending) on 8, 4, 1, 9, 3, the list is:
 - 1, 3, 4, 8, 9
 - 4, 8, 1, 9, 3
 - 1, 8, 4, 9, 3
 - 4, 1, 8, 3, 9
- Binary Search works only on a list that is:
 - sorted
 - unsorted
 - stored in a linked list
 - of strings only

9. Linear Search is also known as:
- a. binary search
 - b. sequential search
 - c. bubble search
 - d. random search
10. How many passes does Bubble Sort need to sort N elements?
- a. N
 - b. $N + 1$
 - c. $N - 1$
 - d. $2N$
11. In Selection Sort pseudocode, the inner loop starts at:
- a. 0
 - b. $N - 1$
 - c. outer - 1
 - d. outer + 1
12. The worst-case time complexity of Binary Search is:
- a. $O(\log N)$
 - b. $O(N)$
 - c. $O(N^2)$
 - d. $O(1)$
13. About how many steps does Binary Search need, at most, to find an item among 1 million sorted elements?
- a. 2
 - b. 20
 - c. 1,000
 - d. 1,000,000
14. Which sorting algorithm is NOT stable?
- a. Bubble Sort
 - b. Both are stable
 - c. Selection Sort
 - d. Neither sorts data
15. Which of the following is NOT an evaluation criterion of an algorithm?
- a. Correctness
 - b. Efficiency
 - c. Clarity
 - d. Popularity

Practice: short questions

1. Define computational thinking and name its four components.
2. Write any four points on the importance of algorithms.
3. Give three differences between an algorithm and pseudocode.
4. What is decomposition? Explain it with the example of learning a new language.
5. What three questions should you ask during pattern recognition? Apply them to the triangle of stars.
6. Define abstraction. In the "making tea" example, name two variables and two noise items.
7. Differentiate between sorting and searching.
8. Why does Binary Search require a sorted list? State two disadvantages of Binary Search.
9. Give two differences between Bubble Sort and Selection Sort.
10. Name the three evaluation criteria of an algorithm. What is a trace table used for?

Practice: long / problem questions

1. Sort the list **5, 2, 7, 1, 6** in ascending order using Bubble Sort. Show every comparison of every pass in a table, and state the total number of comparisons and swaps. Then write the Python code for Bubble Sort.

- Sort the list **6, 2, 9, 5, 1** in ascending order using Selection Sort (textbook method). Show every comparison of every pass and count the swaps. Also state why Selection Sort is called "blind" to an already sorted list.
- The sorted list is **3, 8, 12, 17, 23, 31, 40, 52, 66** (Python indexes 0–8). Using Binary Search, show a low / mid / high table to search for (a) 40 and (b) 20. For each, state how many comparisons Linear Search would need.
- Write the algorithm and pseudocode for Linear Search. Dry-run it on **8, 3, 6, 1, 7, 2, 4, 5** for FindValue = 2 and for FindValue = 9, and give two advantages and two disadvantages of Linear Search.
- A school wants a program that reads the marks of 5 subjects for every student and prints each student's total, percentage and grade. Apply decomposition, pattern recognition and abstraction to this problem, and write a short algorithm.
- A library has 50,000 book IDs stored in sorted order, and the librarian searches it hundreds of times a day. (a) Which search algorithm should be used and why? (b) What is the maximum number of comparisons for Linear and for Binary Search? (c) Explain the three evaluation criteria you would use to judge the chosen program.

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. c | 7. d | 10. c | 13. b |
| 2. d | 5. b | 8. a | 11. d | 14. c |
| 3. a | 6. c | 9. b | 12. a | 15. d |

Short questions — model answers

- Computational thinking is a systematic approach to solving problems. Its four components are decomposition (breaking down), pattern recognition (finding repetitions), abstraction (hiding unnecessary details) and algorithms (writing step-by-step solutions).
- Algorithms (1) provide a structured approach that makes complex tasks manageable; (2) optimise time and memory usage, crucial for large applications; (3) can be reused for similar problems; (4) ensure accuracy and consistency, reducing human error; (5) act as a blueprint before coding. (Any four.)
- (1) An algorithm is a step-by-step logical procedure; pseudocode is a human-readable representation of it using English and keywords. (2) An algorithm is conceptual and language-independent; pseudocode is semi-formal and structured but not executable. (3) An algorithm has no fixed format and is for anyone; pseudocode uses IF/FOR/WHILE, follows conventions and is mainly for programmers.
- Decomposition is breaking a complex task into smaller, manageable parts. Learning a language is decomposed into: vocabulary (words like "I", "eat", "apple"), grammar rules (Subject → Verb → Object), tenses ("I eat" vs "I ate") and sentence building (combining words with rules).
- Ask: What is repeating? What changes in a predictable way? Can I generalise this? For the star triangle: stars repeat in every row; each row has one more star than the previous; generalisation: Row N = N stars, so one loop can print it.
- Abstraction means hiding unnecessary features and focusing on the essential ones. Variables: type of tea (black, green, milk), additions (sugar, lemon), tool (stove or kettle) — any two. Noise: brand of kettle, cup colour or design, where tea is made — any two.
- Sorting arranges a collection of items in ascending or descending order to organise data (e.g., Bubble Sort, Selection Sort). Searching locates a specific element in a list to find its position or whether it exists (e.g., Linear Search, Binary Search).
- Binary Search decides to go left or right by comparing with the middle value; this decision is only valid if all smaller values are on the left and larger on the right, i.e., the list is sorted. Disadvantages: needs sorted

data; needs random access (arrays, not linked lists); code is more complex and prone to off-by-one errors. (Any two.)

9. (1) Bubble Sort compares adjacent elements; Selection Sort compares a fixed position with all later elements. (2) Bubble Sort moves the largest to the end each pass; Selection Sort moves the smallest to the front. Also: Bubble Sort is stable, Selection Sort is not; Selection Sort makes fewer swaps.
10. Correctness, efficiency (time and space) and clarity (readability). A trace table records the values of variables step by step during a dry run, so we can check that the output matches the expected result and so prove correctness.

Long questions — model answers

1. Bubble Sort on 5, 2, 7, 1, 6

Pass	Pair	Left > Right?	List after
1	[5, 2]	Yes, swap	2, 5, 7, 1, 6
1	[5, 7]	No	2, 5, 7, 1, 6
1	[7, 1]	Yes, swap	2, 5, 1, 7, 6
1	[7, 6]	Yes, swap	2, 5, 1, 6, 7
2	[2, 5]	No	2, 5, 1, 6, 7
2	[5, 1]	Yes, swap	2, 1, 5, 6, 7
2	[5, 6]	No	2, 1, 5, 6, 7
3	[2, 1]	Yes, swap	1, 2, 5, 6, 7
3	[2, 5]	No	1, 2, 5, 6, 7
4	[1, 2]	No	1, 2, 5, 6, 7

Total comparisons = 4 + 3 + 2 + 1 = 10; total swaps = 3 + 1 + 1 + 0 = 5. Sorted list: 1, 2, 5, 6, 7.

```
data = [5, 2, 7, 1, 6]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
print(data)
```

OUTPUT

[1, 2, 5, 6, 7]

2. Selection Sort (textbook method) on 6, 2, 9, 5, 1

Pass	Compare	Values	Action	List after
1	Pos1 vs Pos2	6 > 2	Swap	2, 6, 9, 5, 1
1	Pos1 vs Pos3	2 > 9	No	2, 6, 9, 5, 1
1	Pos1 vs Pos4	2 > 5	No	2, 6, 9, 5, 1
1	Pos1 vs Pos5	2 > 1	Swap	1 , 6, 9, 5, 2
2	Pos2 vs Pos3	6 > 9	No	1, 6, 9, 5, 2
2	Pos2 vs Pos4	6 > 5	Swap	1, 5, 9, 6, 2
2	Pos2 vs Pos5	5 > 2	Swap	1, 2 , 9, 6, 5
3	Pos3 vs Pos4	9 > 6	Swap	1, 2, 6, 9, 5
3	Pos3 vs Pos5	6 > 5	Swap	1, 2, 5 , 9, 6
4	Pos4 vs Pos5	9 > 6	Swap	1, 2, 5, 6, 9

10 comparisons, 7 swaps. Sorted list: 1, 2, 5, 6, 9. (With the classic one-swap-per-pass method the passes give 1, 2, 9, 5, 6 → 1, 2, 9, 5, 6 → 1, 2, 5, 9, 6 → 1, 2, 5, 6, 9.)

Selection Sort is "blind" to a sorted list because its loops have fixed limits: it always makes all $N(N-1)/2$ comparisons, even if the list is already in order. It has no way to notice that no swaps were needed and stop early.

3. (a) Search for 40

Step	low	high	mid	data[mid]	Compare	Action
1	0	8	$(0+8)//2 = 4$	23	40 > 23	low = 5
2	5	8	$(5+8)//2 = 6$	40	40 = 40	Found at index 6

Binary: 2 comparisons. Linear Search: 40 is the 7th item, so 7 comparisons.

(b) Search for 20

Step	low	high	mid	data[mid]	Compare	Action
1	0	8	4	23	20 < 23	high = 3
2	0	3	$(0+3)//2 = 1$	8	20 > 8	low = 2
3	2	3	$(2+3)//2 = 2$	12	20 > 12	low = 3
4	3	3	3	17	20 > 17	low = 4
—	4	3	low > high, so the loop stops: Not Found			

Binary: 4 comparisons. Linear Search must check all 9 items before saying "Not Found".

4. Algorithm and pseudocode: as in section 2.4.2 (Start; DATA, N, FindValue, ValueFound := 0; repeat for PTR = 1 to N: if DATA[PTR] = FindValue then ValueFound = 1 and exit; after the loop print DATA[PTR] if ValueFound = 1, else "Not Found"; End).

FindValue = 2:

Step	1	2	3	4	5	6
Element	8	3	6	1	7	2
== 2?	No	No	No	No	No	Yes — exit

Found at position 6 after 6 comparisons; ValueFound = 1; prints 2.

FindValue = 9: 8, 3, 6, 1, 7, 2, 4, 5 are all checked (8 comparisons), none equals 9, ValueFound stays 0, so "Data Not Found" is printed.

Advantages: works on sorted and unsorted data; very simple code; fast for small lists; no extra space.

Disadvantages: up to N checks (1 million for 1 million items); time grows linearly; inefficient if the same list is searched thousands of times. (Any two of each.)

5. **Decomposition:** (1) input a student's 5 marks; (2) calculate total; (3) calculate percentage = total ÷ 500 × 100; (4) decide grade; (5) print result; (6) repeat for the next student.

Pattern recognition: the same steps repeat for every student, and the same "add a mark" step repeats for every subject — so use a loop over students and a loop over subjects.

Abstraction: keep only roll number, marks, total, percentage and grade; ignore noise such as the student's address, uniform or the colour of the result card. The number of students can be a variable.

Algorithm: Step 1: Start. Step 2: Read number of students S. Step 3: Repeat Steps 4–8 for each student. Step 4: TOTAL = 0. Step 5: Repeat 5 times: read MARK, TOTAL = TOTAL + MARK. Step 6: PER = TOTAL / 500 × 100. Step 7: If PER ≥ 80 grade "A1", else if PER ≥ 70 "A", else if PER ≥ 60 "B", else if PER ≥ 50 "C", else "Fail" (school's own grading scale). Step 8: Print TOTAL, PER, GRADE. Step 9: End.

6. (a) **Binary Search**, because the data is already sorted, it is large (50,000 items) and it is searched many times a day, so the time saved per search adds up; Linear Search would be far slower.

(b) Linear Search worst case: 50,000 comparisons. Binary Search worst case: 16 comparisons, because $2^{15} = 32,768 < 50,000 \leq 2^{16} = 65,536$, so 16 halvings are enough to reduce 50,000 items to one.

(c) **Correctness:** it must return the right position for IDs that exist, say "Not Found" for missing IDs (edge cases: first ID, last ID, missing ID), and always stop. **Efficiency:** time $O(\log N)$ and no extra memory ($O(1)$).

Clarity: meaningful names like low, mid, high, clear steps and good indentation so another programmer can maintain it.

Self-assessment: I can...

- | | |
|---|---|
| ☐ define computational thinking and name its four parts. | ☐ dry-run Bubble Sort pass by pass and count comparisons and swaps. |
| ☐ define an algorithm and give five reasons it is important. | ☐ dry-run Selection Sort pass by pass and explain why it is not stable. |
| ☐ draw a table comparing algorithm and pseudocode on at least five points. | ☐ trace Linear Search with a step table, including the "not found" case. |
| ☐ decompose a real-life problem into sub-tasks and explain three benefits. | ☐ trace Binary Search with a BEG / MID / END table and explain why it needs sorted data. |
| ☐ find a pattern in data and turn it into a loop. | ☐ state best and worst time complexity for all four algorithms. |
| ☐ separate necessary steps, variables and noise using abstraction. | ☐ evaluate an algorithm for correctness, efficiency and clarity, and choose the right algorithm for a situation. |

Programming Fundamentals

In this chapter you start writing real programs in Python. You will learn how programs store data, take input, make decisions, repeat work, use ready-made libraries, and how to find and fix your own mistakes.

اس باب میں آپ پاتھن (Python) میں حقیقی پروگرام لکھنا سیکھیں گے۔ آپ جانیں گے کہ پروگرام ڈیٹا کیسے محفوظ کرتا ہے، فیصلے کیسے کرتا ہے، کام کو کیسے دہراتا ہے، اور غلطیاں کیسے تلاش کی جاتی ہیں۔

What you will learn

- Define a program and a programming language, and compare low-, mid- and high-level languages.
- Name career areas where Python is used.
- Describe the features of an IDE and the main parts of the VS Code window.
- Write Python programs that use comments, correct indentation, valid variable names and the main data types.
- Take input with `input()`, convert it with `int()` / `float()`, and display output with `print()` and f-strings.
- Evaluate expressions with arithmetic, assignment, relational, logical, membership and bitwise operators, including binary working.
- Use sequence, selection (`if`, `if-else`, `if-elif-else`, nested `if`) and repetition (`for`, `while`, `range()`, `break`, `continue`, nested loops).
- Import and use built-in libraries (`math`, `random`, `datetime`) and install third-party libraries with `pip`.
- Identify syntax, runtime and logical errors and read a traceback to fix a bug.

3.1 Programming

LEARN IT

Computers are "dumb" machines. They are very fast, but they cannot think or do anything on their own. They need exact, step-by-step **instructions**.

- A set of instructions given to a computer is called a **program**.
- The language used to write these instructions is called a **programming language**.
- Writing programs is called **programming**, and the person who writes them is a **programmer**.

Think of a recipe. A cook follows the recipe line by line: "boil water, add rice, wait 15 minutes". A computer follows a program in the same way, one instruction at a time, without guessing what you meant.

WORKED EXAMPLE

A tiny program. Here is a complete Python program with one instruction. It tells the computer to show a message on the screen.

```
print("Hello")
```

OUTPUT
Hello

The word `print` is the instruction; `"Hello"` is the data it works on.

REMEMBER IT

Program = recipe, programmer = recipe writer, computer = cook who follows every line exactly. If the recipe is wrong, the food is wrong. The cook does not fix it for you.

COMMON MISTAKE

Students often think the computer "understands" what they want. It does not. If you type `Print("Hello")` with a capital P, Python stops with an error, because it only knows `print`. Be exact.

ارو میں سمجھیں
کمپیوٹر خود سے کچھ نہیں سوچ سکتا۔ اسے کام کرنے کے لیے ہدایات چاہئیں۔ ہدایات کے مجموعے کو پروگرام (Program) کہتے ہیں، اور جس زبان میں یہ ہدایات لکھی جاتی ہیں اسے پروگرامنگ لینگویج (Programming Language) کہتے ہیں۔

CHECK YOURSELF

1. What is a program?
2. Why do we say computers are "dumb" machines?

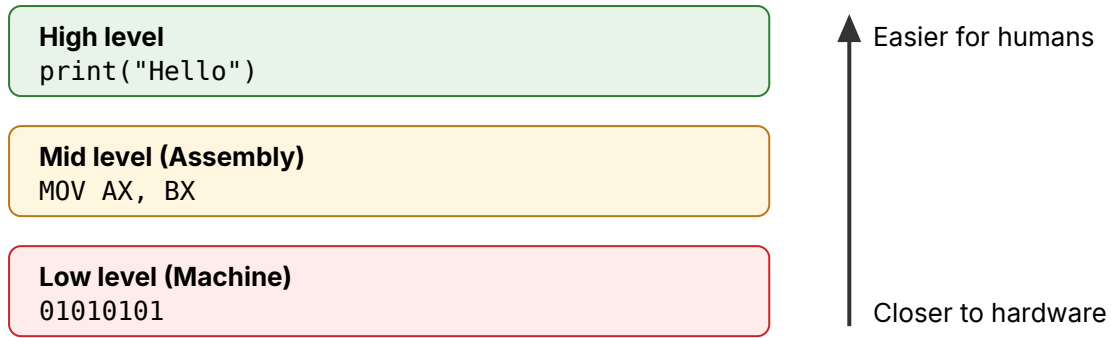
Answers: 1. A set of instructions given to a computer to perform a task. 2. Because they cannot think or do anything without instructions; they only follow exactly what they are told.

3.1.1 Programming Language Types

LEARN IT

Programming languages are classified into **three main types**, depending on how close they are to the computer's own language (binary 0s and 1s) or to human language.

Type	Description	Examples
Low level	Closest to the native language of the computer (binary 0s and 1s). Very difficult for humans to read and write.	Machine language (binary)
Mid level	Bridges the gap. It can access hardware but is easier to read than binary because it uses short words called mnemonics (e.g. <code>MOV</code> , <code>ADD</code>).	Assembly language
High level	Designed to be easy for humans to read and write. Uses English-like words.	C++, Java, Python



The three levels of programming languages, with a sample line from each.

Common high-level languages

Language	Key idea	Used for
C++	Powerful and closer to hardware.	System software, operating systems, high-performance computer games.
Java	"Write once, run anywhere."	Large enterprise applications, Android mobile apps, server programming.
Python	Simple, readable syntax.	Data Science, Artificial Intelligence, Web Development, and learning to program.

WORKED EXAMPLE

Same task, three levels. Suppose you want to copy a value.

- Machine language: a pattern like `01010101` — you would need a chart to know what it means.
- Assembly: `MOV AX, BX` — "move the value in register BX into register AX". Shorter words, but you still need to know the hardware.
- Python: `a = b` — you simply say "a gets the value of b".

The higher the level, the less you need to know about the hardware and the faster you can write the program.

REMEMBER IT

"Low = Like the machine, High = Human." Mid level sits in the Middle and uses Mnemonics (both start with M).

COMMON MISTAKE

"Low level" does not mean "bad" or "weak". Low-level code runs very fast and controls hardware directly. It is just harder for humans to write.

اردو میں سمجھیں

پروگرامنگ زبانوں کی تین اقسام ہیں: لو لیول (Low Level) جو بائنری 0 اور 1 میں ہوتی ہے، مڈ لیول (Mid Level) یعنی اسمبلی جو مختصر الفاظ (Mnemonics) استعمال کرتی ہے، اور ہائی لیول (High Level) جیسے پائتھن، جاوا اور سی پلس پلس جو انگریزی جیسے الفاظ استعمال کرتی ہیں اور انسانوں کے لیے آسان ہیں۔

CHECK YOURSELF

1. Which level is `MOV AX, BX` ?
2. Which high-level language is famous for "write once, run anywhere"?
3. Give one use of C++.

Answers: 1. Mid level (assembly language). 2. Java. 3. Any one: system software, operating systems, high-performance games.

3.2 Career Growth with Python

LEARN IT

Python is one of the most in-demand languages in the world. Learning it opens many career paths:

- **Software Development:** build desktop and web applications.
- **Data Science and Data Analysis:** analyse data, create reports and make predictions.
- **Artificial Intelligence and Machine Learning:** develop intelligent systems, chatbots and recommendation algorithms (like "videos you may like").
- **Web Development:** use frameworks such as **Django** and **Flask** to build websites.
- **Automation and Scripting:** automate repetitive office or lab tasks (renaming 500 files, filling spreadsheets).
- **Cybersecurity and Networking:** write security tools and analyse network data.

Why Python? Python lets you focus on **problem-solving and logic** instead of memorising complex syntax rules. Compare printing a message in Python with C++:

```
# Python
print("Hello")
```

```
// C++
#include <iostream>
int main() {
    std::cout << "Hello";
    return 0;
}
```

WORKED EXAMPLE

Automation example. A school clerk must write "Roll No. 1" to "Roll No. 5" on a list. Python does it in two lines:

```
for n in range(1, 6):
    print("Roll No.", n)
```

OUTPUT

```
Roll No. 1
Roll No. 2
Roll No. 3
Roll No. 4
Roll No. 5
```

Change `6` to `501` and it writes 500 lines just as fast. (You will learn loops in 3.5.3.)

REMEMBER IT

Six careers — remember "**S-D-A-W-A-C**": Software, Data, AI, Web, Automation, Cybersecurity.

COMMON MISTAKE

Django and Flask are **web frameworks** (for websites), not AI tools. AI/ML uses libraries such as TensorFlow and PyTorch (see 3.6.2).

اردو میں سمجھیں

پاتھن سیکھ کر آپ سافٹ ویئر ڈیولپمنٹ، ڈیٹا سائنس، مصنوعی ذہانت (AI)، ویب ڈیولپمنٹ، آٹومیشن اور سائبر سیکیورٹی جیسے شعبوں میں کیریئر بنا سکتے ہیں۔ پاتھن کا آسان انداز آپ کو مشکل اصول یاد کرنے کے بجائے مسئلہ حل کرنے پر توجہ دینے دیتا ہے۔

CHECK YOURSELF

1. Name two Python web frameworks.
2. Which career area builds chatbots and recommendation systems?

Answers: 1. Django and Flask. 2. Artificial Intelligence and Machine Learning.

3.3 Integrated Development Environment (IDE)

LEARN IT

An **IDE (Integrated Development Environment)** is a software tool that helps programmers **write, run and manage code in one place**. It is a complete workspace for programming. "Integrated" means many tools are joined together in one window.

Key features of an IDE:

- **Code Editor:** where you write and edit code.
- **Syntax Highlighting:** colours different parts of the code (keywords, strings, numbers) to make it easier to read.
- **Auto-completion:** suggests code while you type (type `pri` and it offers `print`).
- **Run/Execute Tool:** runs the program with one click.
- **Debugger:** helps you find and fix errors, for example by pausing the program at a chosen line.

WORKED EXAMPLE

Without vs with an IDE. Without an IDE you would write code in Notepad, save it, open a terminal and type `python file.py`, then go back to Notepad to fix mistakes. With an IDE you write, run (press the ► Run button) and debug in the same window, and errors are underlined while you type.

REMEMBER IT

IDE features — "**E-S-A-R-D**": Editor, Syntax highlighting, Auto-completion, Run, Debugger.

COMMON MISTAKE

An IDE does not write the program for you and does not catch logical mistakes. It only makes writing, running and debugging easier.

اردو میں سمجھیں

آئی ڈی ای (IDE) ایک ایسا سافٹ ویئر ہے جس میں کوڈ لکھنا، چلانا اور غلطیاں درست کرنا ایک ہی جگہ ممکن ہوتا ہے۔ اس کی اہم خصوصیات میں کوڈ ایڈیٹر، رنگین کوڈ (Syntax Highlighting)، خودکار تجاویز (Auto completion)، رن بٹن اور ڈیباگر شامل ہیں۔

CHECK YOURSELF

1. What does "integrated" mean in IDE?
2. Which IDE feature suggests code while you type?

Answers: 1. Many programming tools (editor, runner, debugger) are combined in one program. 2. Auto-completion.

3.3.1 Visual Studio Code (VS Code)

LEARN IT

VS Code is a **free, lightweight and powerful code editor** made by Microsoft and widely used for Python. By default it is **not a full IDE**, but it becomes one when you install **extensions** (add-ons), such as the **Python extension by Microsoft**.

VS Code interface elements

Element	Description
1. Top Menu Bar	Control over files, editing, navigation and workspace (File, Edit, Selection, View, Go, Run).
2. Side Bar	Contains the core tools: Explorer, Search, Source Control, Run & Debug and Extensions.
3. Explorer	Shows project files and folders. Helps you create, open and manage your Python files.
4. Search	Finds and replaces text in the open file or across the whole project.
5. Source Control	Connects with Git (a version control system) to track changes, undo mistakes and collaborate.
6. Run & Debug	Runs code, finds errors, pauses at specific lines and lets you inspect variables.
7. Extensions	Like an App Store for VS Code. Add features such as the Python extension, Prettier or Python Debugger.



A simplified VS Code window: menu bar, side bar, Explorer and the code editor.

WORKED EXAMPLE

Setting up and running your first program in VS Code:

Install Python → Install VS Code → Extensions: install "Python" (Microsoft) →

Explorer: new file hello.py → Type code → Click ▶ Run

```
print("Hello, World!")
```

OUTPUT

Hello, World!

The output appears in the **Terminal** panel at the bottom of the window.

REMEMBER IT

Extensions are the **"App Store"** of VS Code: just as apps give your phone new abilities, extensions give VS Code new abilities.

COMMON MISTAKE

Always save Python files with the `.py` extension (e.g. `hello.py`). Without it, VS Code does not know the file is Python, so there is no syntax highlighting and no Run button.

اردو میں سمجھیں

وی ایس کوڈ (VS Code) ایک مفت اور ہلکا کوڈ ایڈیٹر ہے۔ یہ اپنے آپ میں مکمل آئی ڈی ای نہیں، لیکن ایکسٹینشنز (Extensions) لگانے سے آئی ڈی ای بن جاتا ہے۔ اس کے اہم حصے مینو بار، سائیڈ بار، ایکسپلورر، سرچ، سورس کنٹرول، رن اینڈ ڈیباگ اور ایکسٹینشنز ہیں۔

CHECK YOURSELF

1. Which VS Code element connects to Git?
2. Is VS Code a full IDE by default? How does it become one?
3. Where do you see and manage your project's files?

Answers: 1. Source Control. 2. No; it becomes an IDE by installing extensions (e.g. the Python extension). 3. Explorer.

3.4 Fundamentals of Python Programming ★ Golden

LEARN IT

The basic structure of a Python program is simple and clear. A program takes data, **processes it using statements**, and finally **shows output**. Python runs the statements **top to bottom, one after another** (sequentially).

A **statement** is one complete instruction, usually written on one line, such as `print("Hi")` or `x = 5`.

1. Comments (#) — Comments are notes written in the code **for humans**. They start with the `#` symbol. The Python interpreter completely **ignores** them. Use them to explain what your code does.

2. Indentation (spaces) — C++ and Java use curly brackets `{ }` to mark a **block** of code (a group of statements that belong together). Python instead uses **indentation**: spaces at the beginning of a line. Indentation is **mandatory** in Python. The standard is **4 spaces** (or 1 tab).

```
# This is a comment. It explains the code below.  
print("Hello, World!")  
if 5 > 2:  
    # The space before 'print' is the indentation  
    print("Five is greater than two!")
```

OUTPUT

```
Hello, World!  
Five is greater than two!
```

WORKED EXAMPLE

Reading the program line by line:

1. Line 1 starts with #, so Python skips it.
2. Line 2 prints Hello, World!.
3. Line 3 asks a question: is 5 greater than 2? The colon : says "a block follows".
4. Line 4 is a comment inside the block (ignored).
5. Line 5 is indented by 4 spaces, so it belongs to the if block. Because $5 > 2$ is true, it runs.

What happens if you forget the indentation?

```
if 5 > 2:  
print("Five is greater than two!")
```

OUTPUT

```
File "indent.py", line 2  
    print("Five is greater than two!")  
    ^
```

IndentationError: expected an indented block after 'if' statement on line 1

Python refuses to run the program and points to line 2. Add 4 spaces before print to fix it.

REMEMBER IT

Think of indentation like the points and sub-points in your notebook. Sub-points are written a little to the right to show they "belong" to the main point. A colon : at the end of a line always means "indent the next line".

COMMON MISTAKE

IndentationError? Check your spaces. **Mixing tabs and spaces** can break a Python program even when the code looks lined up. Choose 4 spaces and use them everywhere (VS Code inserts 4 spaces when you press Tab in a .py file).

اردو میں سمجھیں

پاتھن پروگرام اوپر سے نیچے ترتیب سے چلتا ہے۔ # سے شروع ہونے والی لائن تبصرہ (Comment) ہوتی ہے جسے پاتھن نظر انداز کر دیتا ہے۔ پاتھن میں کوڈ کے بلاک کو بریکٹ کے بجائے انڈینٹیشن (Indentation) یعنی لائن کے شروع میں چار خالی جگہوں سے ظاہر کیا جاتا ہے، اور یہ لازمی ہے۔

EXAM TIP

Common questions: "What is the purpose of comments?", "Why is indentation important in Python?", "How is Python different from C++/Java in defining blocks?" Write: comments start with # and are ignored by the interpreter; Python uses indentation (4 spaces) instead of { }; wrong indentation gives an IndentationError. Always give a 3–4 line code example.

CHECK YOURSELF

1. Predict the output:

```
# print("A")
print("B")
if 3 > 7:
    print("C")
print("D")
```

2. What is the standard amount of indentation in Python?

Answers: 1. B then D on separate lines (line 1 is a comment; 3 > 7 is false so C is skipped). 2. 4 spaces (or 1 tab).

3.4.1 Variables ★ Golden

LEARN IT

A **variable** is a **named space in computer memory used to store data**. Think of it as a labelled container (a box with a name written on it). You put a value in it with the **assignment operator** `=`, which means "store", not "equals".

In Python you **do not need to declare the type** of a variable. Python figures it out automatically from the value. This is called **dynamic typing**: the same variable can later hold a different type.

Rules for naming variables:

- Can contain **letters, numbers and underscores** (`_`) only — no spaces or symbols like `-`, `@`, `$`.
- **Must start with a letter or underscore**, NOT a number (`1name` is invalid).
- Are **case-sensitive**: `age` and `Age` are two completely different variables.
- Cannot be a Python **keyword** (a reserved word such as `if`, `for`, `while`, `else`, `True`).

Valid names ✓

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
```

Invalid names ✗

```
1name = "Ali"    # starts with a number
my height = 5.7  # contains a space
if = 100         # keyword used
```

WORKED EXAMPLE

Using variables and case-sensitivity:

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
print(name, age, my_height, score1)

Age = 17          # a different variable from age
print(age, Age)
```

OUTPUT

```
Ali 16 5.7 100
16 17
```

Dynamic typing — the type follows the value. `type()` tells you the type:

```
x = 5
print(x, type(x))
x = "five"
print(x, type(x))
```

OUTPUT

```
5 <class 'int'>
five <class 'str'>
```

What Python says about an invalid name:

```
1name = "Ali"
```

OUTPUT

```
File "names.py", line 1
    1name = "Ali"
      ^
SyntaxError: invalid decimal literal
```

REMEMBER IT

Choose **meaningful names**: `total_marks` is far clearer than `tm` or `x`. Use lowercase words joined by underscores (this style is called **snake_case**).

COMMON MISTAKE

`=` is **not** "equals" in the maths sense. `x = x + 1` is not a false equation; it means "take the value of `x`, add 1, and store the result back in `x`". To **compare** two values you use `==` (see 3.4.3).

اردو میں سمجھیں

ویری ایبل (Variable) میموری میں ایک نام والی جگہ ہے جس میں ڈیٹا رکھا جاتا ہے، بالکل ایک لیبل لگے ڈبے کی طرح۔ پاتھن میں ٹائپ بتانے کی ضرورت نہیں ہوتی۔ نام حرف یا انڈر سکور سے شروع ہونا چاہیے، اس میں خالی جگہ نہیں ہو سکتی، یہ کیس سینسیٹو (Case-sensitive) ہوتا ہے اور کوئی کی ورڈ (Keyword) نہیں ہو سکتا۔

EXAM TIP

A favourite question: "Which of these variable names are valid? Give reasons." Check each name against all four rules and state the rule broken, e.g. "2nd_place — invalid, starts with a digit". Remember that `While` (capital W) is **valid** because keywords are lowercase and Python is case-sensitive.

CHECK YOURSELF

1. Valid or invalid? `_total`, `roll-no`, `class`, `marks2026`, `2marks`
2. Predict the output:

```
city = "Karachi"
City = "Hyderabad"
print(city)
```

Answers: 1. `_total` valid; `roll-no` invalid (hyphen); `class` invalid (keyword); `marks2026` valid; `2marks` invalid (starts with a digit). 2. `Karachi` — `city` and `City` are different variables.

3.4.1 Data Types in Python ★ Golden

LEARN IT

A **data type** specifies the **kind of data** a variable holds — a whole number, a decimal, text, True/False, or a collection of items. The type decides what you can do with the data (you can divide numbers, but not names). Python has several built-in data types:

Name	Type	Description	Example
Integer	<code>int</code>	Whole numbers without decimals (positive, negative or zero).	<code>age = 16</code>
Floating point	<code>float</code>	Numbers with a decimal point.	<code>height = 5.7</code>
String	<code>str</code>	Text data, enclosed in quotes (" " or ' ').	<code>name = "Sammy"</code> or <code>year = "2026"</code>
Boolean	<code>bool</code>	Logical value: <code>True</code> or <code>False</code> (capital T / F).	<code>is_student = True</code>
List	<code>list</code>	Ordered, changeable collection in square brackets [].	<code>marks = [85, 90, 75]</code>
Tuple	<code>tuple</code>	Ordered, unchangeable sequence in parentheses ().	<code>coords = (10, 20)</code>
Dictionary	<code>dict</code>	Key : value pairs in curly braces { }.	<code>student = {"name": "Ahmed", "age": 16}</code>
Set	<code>set</code>	Unordered collection of unique items (no duplicates).	<code>vowels = {'a', 'e', 'i'}</code>

Note: the slide abbreviates tuple as "tup"; the real Python type name is `tuple`, as the program below shows. The slides describe dictionaries as unordered; since Python 3.7 a dictionary remembers the order in which keys were added, but you always look items up **by key**, not by position.

WORKED EXAMPLE

Checking every type with `type()` :

```
age = 16
height = 5.7
name = "Sammy"
year = "2026"
is_student = True
marks = [85, 90, 75]
coords = (10, 20)
student = {"name": "Ahmed", "age": 16}
vowels = {'a', 'e', 'i'}

print(type(age))
print(type(height))
print(type(name), type(year))
print(type(is_student))
print(type(marks))
print(type(coords))
print(type(student))
print(type(vowels))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'> <class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

Notice that `year = "2026"` is a **string**, not a number, because it is inside quotes.

Working with the collection types:

```
marks = [85, 90, 75]
marks[0] = 88          # lists are changeable
marks.append(60)
print(marks)

coords = (10, 20)
print(coords[0])       # read item at position 0

student = {"name": "Ahmed", "age": 16}
print(student["name"]) # look up by key

nums = {1, 2, 2, 3, 3, 3} # duplicates are removed
print(nums)
print(len(nums))
```

OUTPUT

```
[88, 90, 75, 60]
10
Ahmed
{1, 2, 3}
3
```

Positions (called **indexes**) start at **0**, so `coords[0]` is the first item. Trying to change a tuple causes an error:

```
coords = (10, 20)
coords[0] = 50
```

OUTPUT

```
Traceback (most recent call last):
  File "tuple_test.py", line 2, in <module>
    coords[0] = 50
    ~~~~~^
TypeError: 'tuple' object does not support item assignment
```

REMEMBER IT

Brackets decide the collection: `[ ]` List (changeable), `( )` Tuple (locked), `{ }` with **key:value** = Dictionary, `{ }` with single items = Set (unique). Think: a **list** is a shopping list you can edit; a **tuple** is your date of birth — it never changes.

COMMON MISTAKE

`True` and `False` must start with a capital letter. `true` is not a Boolean; Python treats it as an unknown variable name and gives a `NameError`. Also, `"16"` (string) and `16` (int) are different: `"16" + "1"` gives `"161"`, while `16 + 1` gives `17`.

اردو میں سمجھیں

ڈیٹا ٹائپ (Data Type) بتاتی ہے کہ ویری ایبل میں کس قسم کا ڈیٹا ہے۔ `int` مکمل اعداد، `float` اعشاریہ والے اعداد، `str` متن، اور `bool` صرف `True` یا `False` کے لیے ہے۔ لسٹ (List) تبدیل ہو سکتی ہے، ٹپل (Tuple) تبدیل نہیں ہو سکتا، ڈکشنری (Dictionary) میں کی اور ویلیو کے جوڑے ہوتے ہیں، اور سیٹ (Set) میں کوئی چیز دہرائی نہیں جاتی۔

EXAM TIP

Expect: "Name any four data types with examples" or "Differentiate list and tuple". For a difference question, use a two-column table: list — `[ ]`, changeable, e.g. `marks = [85, 90]`; tuple — `( )`, unchangeable, e.g. `coords = (10, 20)`.

CHECK YOURSELF

1. What is the data type of each? `3.0`, `"3"`, `3`, `False`, `[3]`
2. Predict the output: `print(len({5, 5, 6, 6, 6}))`
3. Which type should store a student's CNIC/B-Form number such as `"42101-1234567-1"` ? Why?

Answers: 1. float, str, int, bool, list. 2. 2 (the set keeps only 5 and 6). 3. `str`, because it contains hyphens and you never do arithmetic on it.

3.4.2 Input and Output Handling

LEARN IT

Output — the `print()` function. A **function** is a named, ready-made piece of code that does a job when you call it with brackets. `print()` displays information on the screen.

- Items separated by commas are printed with one space between them.
- An **f-string** (a string with `f` before the quote) lets you put variables directly inside `{ }`. It is the fastest and most readable way to format output.

Input — the `input()` function. It shows a message (a **prompt**), **pauses the program** and waits for the user to type something and press Enter. Whatever the user types is **always a string** by default.

Type casting means converting a value from one type to another: `int()` converts to an integer, `float()` to a decimal number, `str()` to a string.

```
# 1. Print only a string
print("Hello Class!")

# 2. Print a string and a variable (comma separates them)
name = "Ali"
print("Welcome,", name)

# 3. f-strings (fastest and most readable way to format)
age = 16
print(f"My name is {name} and I am {age} years old.")
```

OUTPUT

```
Hello Class!
Welcome, Ali
My name is Ali and I am 16 years old.
```

Two useful extras: `sep` changes the separator between items, and `end` changes what is printed at the end (normally a new line).

```
print("10", "09", "2026", sep="-")
print("Loading", end="...")
print("done")
print(f"Total: {7 * 3}")
```

OUTPUT

```
10-09-2026
Loading... done
Total: 21
```

WORKED EXAMPLE

Taking input with type casting.

```
# Taking text input
user_name = input("Enter your name: ")
# Type casting: converting string input to an integer (int)
age = int(input("Enter your age: "))
# Type casting: converting string input to a decimal (float)
height = float(input("Enter your height: "))
print(f"Hello {user_name}! Next year you will be {age + 1}.")
print("Height:", height, "feet")
```

Sample run (the user types Sara, 16 and 5.4):

OUTPUT

```
Enter your name: Sara
Enter your age: 16
Enter your height: 5.4
Hello Sara! Next year you will be 17.
Height: 5.4 feet
```

Why type casting matters. Without `int()`, the numbers stay as text and `+` joins them:

```
a = input("First number: ")
b = input("Second number: ")
print(a + b)
print(int(a) + int(b))
```

OUTPUT

```
First number: 5
Second number: 3
53
8
```

"5" + "3" joins two strings into "53" (called **concatenation**). 5 + 3 adds numbers to give 8.

REMEMBER IT

"input() always gives a string." If you plan to do maths, wrap it: `int(input(...))` for whole numbers, `float(input(...))` for decimals. Read it from the inside out: first `input`, then convert.

COMMON MISTAKE

If the user types letters where a number is expected, `int()` crashes with `ValueError: invalid literal for int() with base 10` (see 3.7). Also, `int("5.4")` fails — use `float()` for decimal input.

`print()` اسکرین پر معلومات دکھاتا ہے اور `input()` صارف سے معلومات لیتا ہے۔ یاد رکھیں کہ `input()` ہمیشہ اسٹرنگ (String) دیتا ہے، اس لیے حساب کے لیے اسے `int()` یا `float()` سے تبدیل کرنا ضروری ہے۔ اسے ٹائپ کاسٹنگ (Type Casting) کہتے ہیں۔ ایف اسٹرنگ (f-string) سے ویری ایبلز کو جملے کے اندر آسانی سے لکھا جاسکتا ہے۔

CHECK YOURSELF

1. Predict the output:

```
a = "3"
b = 4
print(a * b)
print(int(a) * b)
```

2. Find the error: the user types 21 but the output is 2121, not 42.

```
num = input("Enter a number: ")
print(num * 2)
```

Answers: 1. 3333 then 12 (a string times a number repeats the string). 2. `input()` returns the string "21", and a string times 2 repeats it. Fix: `num = int(input("Enter a number: "))`.

3.4.3 Operators and Operands: Arithmetic and Assignment ★ Golden

LEARN IT

An **operand** is the value or variable being worked on. An **operator** is the symbol that performs the action. Together they form an **expression**, which Python evaluates to a single value.

In `10 + 20`: **10** is an operand, **+** is the operator, **20** is an operand, and the result is **30**.

1. **Arithmetic operators** do maths:

Operator	Purpose	Example	Result
+	Addition	<code>5 + 3</code>	8
-	Subtraction	<code>10 - 4</code>	6
*	Multiplication	<code>4 * 2</code>	8
/	Division (result is always a float)	<code>9 / 2</code>	4.5
//	Floor division (drops the decimal part)	<code>9 // 2</code>	4
%	Modulus (remainder)	<code>9 % 2</code>	1
**	Exponentiation (power)	<code>2 ** 3</code>	8

```

print(5 + 3)
print(10 - 4)
print(4 * 2)
print(9 / 2)
print(9 // 2)
print(9 % 2)
print(2 ** 3)
print(8 / 2)

```

OUTPUT

```

8
6
8
4.5
4
1
8
4.0

```

Notice the last line: `8 / 2` gives `4.0`, not `4`, because `/` always gives a float.

Order of operations (precedence). Python follows the same order as maths (like BODMAS): brackets first, then `**`, then `*` `/` `//` `%` (left to right), then `+` `-` (left to right). `**` is the exception: it works right to left.

```

print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(-9 // 2)
print(17 % 5, 17 // 5)

```

OUTPUT

```

14
20
512
3
-5
2 3

```

`2 ** 3 ** 2 = 2 ** 9 = 512` (right to left). `-9 // 2` is `-5` because floor division always rounds **down** to the smaller whole number ($-4.5 \rightarrow -5$).

2. Assignment operators store a value in a variable, often updating it. The short forms are called **compound assignment**.

Operator	Example	Equivalent to
<code>=</code>	<code>x = 5</code>	Assign 5 to x
<code>+=</code>	<code>x += 2</code>	<code>x = x + 2</code>
<code>-=</code>	<code>x -= 1</code>	<code>x = x - 1</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code> , <code>/=</code> , <code>%=</code> , <code>**=</code>	<code>x /= 2</code>	<code>x = x / 2</code> (and so on)

WORKED EXAMPLE

Tracing assignment operators. Follow the value of `x` line by line:

```
x = 5
x += 2      # x = 5 + 2
print(x)
x -= 1      # x = 7 - 1
print(x)
x *= 3      # x = 6 * 3
print(x)
x /= 2      # x = 18 / 2
print(x)
```

OUTPUT

```
7
6
18
9.0
```

Using `%` and `//` to split digits. `% 10` gives the last digit; `// 10` removes it.

```
n = 357
last = n % 10      # remainder after dividing by 10
rest = n // 10     # drop the last digit
print("Last digit:", last)
print("Remaining:", rest)
```

OUTPUT

```
Last digit: 7
Remaining: 35
```

$357 \div 10 = 35$ remainder 7. So `357 // 10 = 35` and `357 % 10 = 7`.

REMEMBER IT

Three kinds of division: `/` = exact answer (float), `//` = quotient only ("floor"), `%` = remainder only. A number is **even** when `n % 2 == 0`.

COMMON MISTAKE

`^` is **not** power in Python — it is bitwise XOR (see bitwise operators). `2 ^ 3` gives 1, not 8. For power, always use `**`.

اردو میں سمجھیں

آپرینڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔ ہمیشہ اعشاریہ والا جواب دیتا ہے، `//` صرف حاصل تقسیم دیتا ہے، `%` باقی دیتا ہے اور `**` طاقت (Power) نکالتا ہے۔ `x += 2` کا مطلب ہے `x = x + 2`

EXAM TIP

You will be asked to evaluate expressions such as `17 // 5`, `17 % 5`, `2 ** 4` or to trace compound assignment. Show each step and write the final value with its type (e.g. `9.0` is a float, so write the `.0`).

CHECK YOURSELF

1. Predict the output:

```
x = 7
y = 2
print(x // y, x % y, x / y)
print(x ** y)
```

2. Predict the output:

```
x = 10
x += 5
x /= 4
print(x)
```

Answers: 1. `3 1 3.5` then `49` . 2. `3` (`10 + 5 = 15`; `15 // 4 = 3`).

3.4.3 Relational, Logical and Membership Operators ★ Golden

LEARN IT

3. Relational (comparison) operators compare two values and return a **Boolean** (`True` or `False`). They are used in conditions.

Op	Description	Example (a = 10, b = 5)	Result
<code>=</code>	Equal to	<code>a == b</code>	<code>False</code>
<code>≠</code>	Not equal to	<code>a != b</code>	<code>True</code>
<code>></code>	Greater than	<code>a > b</code>	<code>True</code>
<code><</code>	Less than	<code>a < b</code>	<code>False</code>
<code>≥</code>	Greater than or equal to	<code>a ≥ 10</code>	<code>True</code>
<code>≤</code>	Less than or equal to	<code>a ≤ b</code>	<code>False</code>

```
a = 10
b = 5
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a ≥ 10)
print(a ≤ b)
```

OUTPUT

```
False
True
True
False
True
False
```

4. Logical operators combine several conditions:

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

- **and** — True **only if both** conditions are true.
- **or** — True if **at least one** condition is true.
- **not** — **reverses** the result (True becomes False, False becomes True).

```
a = 10
b = 5
print(a > 5 and b > 5)
print(a > 5 or b > 5)
print(not a > 5)
```

OUTPUT

```
False
True
False
```

5. Membership operators check whether a value exists inside a sequence (such as a string or list): **in** and **not in**.

```
name = "Python"
print("P" in name)
print("z" not in name)
print("p" in name)
marks = [85, 90, 75]
print(90 in marks)
```

OUTPUT

```
True
True
False
True
```

"p" in name is False because strings are case-sensitive: the word has a capital P.

WORKED EXAMPLE

Real-life conditions. A person can apply for a CNIC if they are 18 or older **and** have the documents. Marks are invalid if they are below 0 **or** above 100.

```
age = 20
has_cnic = True
print(age ≥ 18 and has_cnic)
marks = 45
print(marks < 0 or marks > 100)
```

OUTPUT

```
True
False
```

Step by step: $20 \geq 18$ is True, `has_cnic` is True, True and True = **True**. For marks: $45 < 0$ is False, $45 > 100$ is False, False or False = **False** (the marks are valid).

REMEMBER IT

AND is strict, OR is generous. AND is like a teacher who needs both homework and attendance; OR is like a shop that accepts cash **or** card. Order of evaluation: `not` first, then `and`, then `or`.

COMMON MISTAKE

Do not confuse `=` (assignment, "store") with `==` (comparison, "is it equal?"). Writing `if x = 5:` is a **syntax error**; the correct form is `if x == 5:`.

ارو میں سمجھیں

ریشنل آپریٹرز (Relational Operators) دو ویلیوز کا موازنہ کر کے True یا False دیتے ہیں۔ لاجیکل آپریٹرز میں `and` تب True دیتا ہے جب دونوں شرطیں درست ہوں، `or` تب جب کم از کم ایک درست ہو، اور `not` نتیجہ الٹ دیتا ہے۔ `in` دیکھتے ہیں کہ کوئی چیز اسٹرنگ یا لسٹ میں موجود ہے یا نہیں۔

EXAM TIP

Typical: "Evaluate `a > b` and `b > 3` for `a = 10`, `b = 5`" or "Write the truth table of `and` / `or`". Write each sub-condition's value first, then combine. Remember: `==` compares, `=` assigns.

CHECK YOURSELF

1. Predict the output:

```
print(True and False or True)
print(not (5 > 3) or 2 == 2.0)
```

2. Find the error:

```
x = 5
if x = 5:
    print("five")
```

Answers: 1. True and True. (True and False = False; False or True = True. not True = False; 2 == 2.0 is True; False or True = True.) 2. = is assignment; a comparison needs ==. Python reports `SyntaxError: invalid syntax`. Maybe you meant '=' or '==' instead of '='? Fix: `if x == 5:`

3.4.3 Bitwise Operators ★ Golden

LEARN IT

6. Bitwise operators treat numbers as **binary digits** (bits: 0s and 1s) and work on them **bit by bit**, column by column. Think of each bit as a switch that is ON (1) or OFF (0). Python's `bin()` function shows the binary form of a number (it starts with `0b`).

- **AND (&)** — gives 1 **only if both** bits are 1, otherwise 0.
- **OR (|)** — gives 1 if **at least one** bit is 1.
- **XOR (^)** — gives 1 if the bits are **different**, 0 if they are the **same**.
- **NOT (~)** — inverts all bits (0 → 1, 1 → 0). In Python the result is the number made negative, minus 1: `~x = -(x + 1)`. So `~10` gives `-11`.
- **Left shift (<<)** — moves bits left and adds 0s on the right. Each shift **multiplies by 2**.
- **Right shift (>>)** — moves bits right and removes the rightmost bits. Each shift **divides by 2** (floor division).

Bit A	Bit B	A & B	A B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

WORKED EXAMPLE

Worked example: a = 10, b = 6. First convert to binary using place values 8, 4, 2, 1: 10 = 8 + 2 = **1010**; 6 = 4 + 2 = **0110**. Then work column by column.

	8	4	2	1	Decimal
a = 10	1	0	1	0	10
b = 6	0	1	1	0	6
a & b	0	0	1	0	2
a b	1	1	1	0	14
a ^ b	1	1	0	0	12

- AND: only the "2" column has 1 in both → 0010 = **2**.
- OR: columns 8, 4 and 2 have at least one 1 → 1110 = 8 + 4 + 2 = **14**.
- XOR: columns 8 and 4 are different; column 2 is the same (both 1) → 1100 = 8 + 4 = **12**.
- NOT: $\sim 10 = -(10 + 1) = \mathbf{-11}$.

```
a = 10
b = 6
print(bin(a), bin(b))
print(a & b)
print(a | b)
print(a ^ b)
print(~a)
```

OUTPUT

```
0b1010 0b110
2
14
12
-11
```

Worked example: shifts. 5 in 8 bits is **0000 0101**.

Expression	Binary	Decimal	What happened
5	0000 0101	5	start
5 << 1	0000 1010	10	add one 0 on the right (5 × 2)
5 << 2	0001 0100	20	add 00 on the right (5 × 4)
20 >> 1	0000 1010	10	remove the rightmost bit (20 ÷ 2)
20 >> 2	0000 0101	5	remove two bits (20 ÷ 4)

```
a = 5          # Binary: 0000 0101
print(a << 1)   # 0000 1010
print(a << 2)   # 0001 0100
b = 20         # Binary: 0001 0100
print(b >> 1)   # 0000 1010
print(b >> 2)   # 0000 0101
print(bin(20 >> 2))
```

OUTPUT

```
10
20
10
5
0b101
```

REMEMBER IT

Shortcuts: $x \ll n = x \times 2^n$, $x \gg n = x // 2^n$, $\sim x = -x - 1$. For AND/OR/XOR always write both numbers in binary, one under the other, with the same number of bits.

COMMON MISTAKE

Do not mix up the logical and bitwise operators: `and` / `or` / `not` work on True/False conditions, while `&` / `|` / `~` work on the bits of integers. Also remember `bin(6)` prints `0b110`, without the leading zero — add zeros yourself when lining up columns.

اردو میں سمجھیں

بٹ وائز آپریٹرز (Bitwise Operators) اعداد کو بائنری میں بدل کر ہر بٹ پر الگ الگ کام کرتے ہیں۔ `&` تب 1 دیتا ہے جب دونوں بٹ 1 ہوں، `|` تب جب کوئی ایک بھی 1 ہو، اور `^` تب جب دونوں بٹ مختلف ہوں۔ `~` تمام بٹ الٹ دیتا ہے، اس لیے `~10` کا جواب 11 ہے۔ لیفٹ شفٹ عدد کو 2 سے ضرب دیتا ہے اور رائٹ شفٹ 2 سے تقسیم کرتا ہے۔

EXAM TIP

Exams ask you to "evaluate `12 & 10`, `12 | 10`, `12 ^ 10`, `~5`, `7 >> 1` showing binary working". Marks are given for the binary conversion and column working, not just the final number. Always draw the column table.

CHECK YOURSELF

1. Predict the output (show the binary working):

```
print(12 & 10, 12 | 10, 12 ^ 10)
print(~5, 7 >> 1, 3 << 3)
```

2. What does `bin(13)` print?

Answers: 1. 8 14 6 and -6 3 24. $12 = 1100$, $10 = 1010$: AND = $1000 = 8$, OR = $1110 = 14$, XOR = $0110 = 6$. $\sim 5 = -(5 + 1) = -6$. $7 = 111 \rightarrow 11 = 3$. $3 = 11 \rightarrow 11000 = 24$ (3×8). 2. `0b1101` ($8 + 4 + 1$).

3.5 Control Structures ★ Golden

LEARN IT

Control structures decide the **order in which statements are executed** in a program (the "flow of control"). Almost every program is built from just three basic structures:

1. **Sequence** — do the steps one after another, top to bottom.
2. **Selection** — choose a path depending on a condition (`if` , `else`).
3. **Repetition** — repeat steps (loops: `for` , `while`).

WORKED EXAMPLE

Morning routine as control structures:

- Sequence: wake up → brush teeth → have breakfast.
- Selection: **if** it is raining, take an umbrella; **else** take a cap.
- Repetition: **while** the bus has not come, keep waiting.

REMEMBER IT

Remember **"S-S-R"**: **S**equence, **S**election, **R**epetition — "Step, Select, Repeat".

COMMON MISTAKE

Selection and repetition both use conditions, but they are different: selection runs a block **at most once**; repetition can run a block **many times**.

اردو میں سمجھیں
کنٹرول اسٹرکچرز (Control Structures) طے کرتے ہیں کہ پروگرام کی ہدایات کس ترتیب سے چلیں گی۔ تین بنیادی اسٹرکچر ہیں:
سیکوننس (ترتیب وار)، سلیکشن (شرط کی بنیاد پر انتخاب) اور ریپیتیشن (دہرانا یعنی لوپ)۔

EXAM TIP

"Name and explain the three basic control structures" is a very common short question. Give a one-line definition and a two-line Python example for each.

CHECK YOURSELF

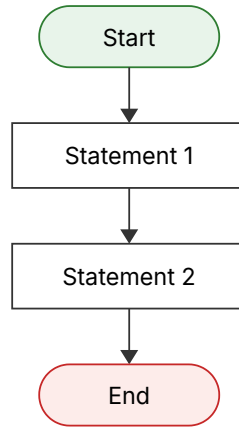
1. Name the three basic control structures.
2. Which structure is used for "keep asking for the password until it is correct"?

Answers: 1. Sequence, selection, repetition. 2. Repetition (a `while` loop).

3.5.1 Sequence

LEARN IT

Sequence means statements are executed **one after another, strictly from top to bottom**, in the exact order they are written. This is the **default behaviour** of Python. No statement is skipped or repeated.



Sequence: control flows straight down from one statement to the next.

WORKED EXAMPLE

Input → Process → Output in sequence.

```

print("Enter two numbers")
a = int(input("Enter first number: ")) # Step 1: Input
b = int(input("Enter second number: ")) # Step 2: Input
total = a + b                         # Step 3: Process
print("Sum =", total)                 # Step 4: Output
  
```

Sample run (user types 7 and 5):

```

OUTPUT
Enter two numbers
Enter first number: 7
Enter second number: 5
Sum = 12
  
```

Each line runs exactly once, in order. If you moved line 4 (`total = a + b`) above the input lines, Python would stop with a `NameError` because `a` and `b` would not exist yet.

REMEMBER IT

Sequence is like reading a page: line 1, then line 2, then line 3. Most programs follow the pattern **Input → Process → Output**.

COMMON MISTAKE

Order matters. A variable must be given a value **before** it is used. Using it first causes `NameError: name '...' is not defined`.

اردو میں سمجھیں

سیکوننس (Sequence) کا مطلب ہے کہ ہدایات بالکل اسی ترتیب سے ایک کے بعد ایک اوپر سے نیچے چلتی ہیں جس ترتیب سے لکھی گئی ہیں۔ یہ پاتھن کا عام طریقہ ہے۔ کسی ویری ایبل کو استعمال کرنے سے پہلے اس میں ویلیو رکھنا ضروری ہے۔

CHECK YOURSELF

1. Predict the output:

```
x = 4
y = x * 2
x = 10
print(x, y)
```

2. What is the default control structure in Python?

Answers: 1. 10 8 — y was calculated when x was still 4; changing x later does not change y. 2. Sequence.

3.5.2 Selection Statements: if and if-else ★ Golden

LEARN IT

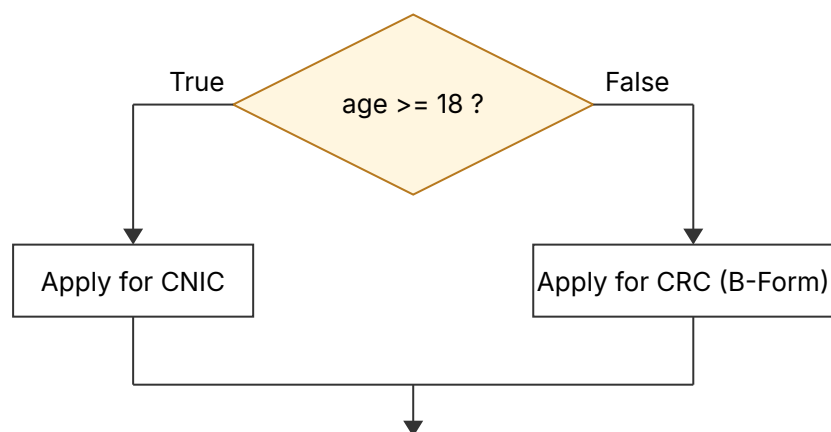
Selection changes the flow of a program based on a **condition** — an expression that is either True or False. It helps a program respond intelligently to different situations.

1. The **if statement** executes a block of code **only if** the condition is True. If the condition is False, the block is completely skipped.

```
if condition:
    statement(s)    # runs only when condition is True
```

2. The **if-else statement** provides **two paths**: if the condition is true, do this; **else** (otherwise), do that. Exactly one of the two blocks always runs.

```
if condition:
    block A        # condition True
else:
    block B        # condition False
```



An if-else statement: one diamond (condition), two paths, then the paths join again.

WORKED EXAMPLE

The if statement.

```
age = int(input("Enter your age: "))
if age ≥ 18:
    print("You are an Adult.")    # runs only if age ≥ 18
print("Program continues...")    # runs every time
```

Run 1 (user types 20):

OUTPUT

```
Enter your age: 20
You are an Adult.
Program continues...
```

Run 2 (user types 15):

OUTPUT

```
Enter your age: 15
Program continues...
```

The last line is not indented, so it is outside the `if` block and runs every time.

The if-else statement.

```
age = int(input("Enter your age: "))
if age ≥ 18:
    print("Apply for CNIC.")
else:
    print("Apply for CRC (B-Form).")
```

Sample run (user types 16):

OUTPUT

```
Enter your age: 16
Apply for CRC (B-Form).
```

Even or odd — a classic if-else program:

```
n = 7
if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

OUTPUT

```
7 is odd
```

$7 \% 2 = 1$, and $1 == 0$ is False, so the `else` block runs.

REMEMBER IT

Formatting rule: put a **colon (:)** after `if` and `else`, and **indent** the code inside the blocks. `else` never has a condition of its own.

COMMON MISTAKE

Common mistakes: forgetting the colon (`SyntaxError: expected ':'`), writing a condition after `else` (`else age < 18:` is wrong), and not indenting `else` at the same level as its `if`.

سلیکشن (Selection) میں پروگرام شرط کی بنیاد پر فیصلہ کرتا ہے۔ if کا بلاک صرف تب چلتا ہے جب شرط درست (True) ہو۔ if-else میں دو راستے ہوتے ہیں: شرط درست ہو تو پہلا بلاک، ورنہ else والا بلاک چلتا ہے۔ if اور else کے بعد کولن (:) اور اندر کے کوڈ کی انڈینٹیشن ضروری ہے۔

EXAM TIP

Asked as: "Write a program that inputs a number and prints whether it is even or odd / positive or negative / pass or fail." Write the full program with `int(input())`, the condition, both blocks, and a sample run.

CHECK YOURSELF

1. Predict the output when `marks = 32`:

```
marks = 32
if marks ≥ 33:
    print("Pass")
else:
    print("Fail")
print("Done")
```

2. Find the two errors:

```
n = 5
if n > 0
    print("Positive")
else n ≤ 0:
    print("Not positive")
```

Answers: 1. Fail then Done . 2. Missing colon after `if n > 0`; else must not have a condition — write `else:` .

3.5.2 Selection Statements: if-elif-else and Nested if

★ Golden

LEARN IT

3. The if-elif-else statement handles **many conditions**. (`elif` is short for "else if".) Python checks the conditions **from top to bottom**. As soon as it finds a True condition, it runs that block and **ignores all the rest**. If none is True, the `else` block runs.

4. Nested if is an `if` inside another `if`. It is used when a decision needs to pass **several levels of checks**: the inner check happens only if the outer check passed.

WORKED EXAMPLE

Grading with if-elif-else.

```
marks = int(input("Enter marks: "))
if marks ≥ 80:
    print("A-1 Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 60:
    print("B Grade")
else:
    print("Fail or below B")
```

Sample run (user types 75):

OUTPUT

```
Enter marks: 75
A Grade
```

Trace for different marks:

marks	≥ 80?	≥ 70?	≥ 60?	Output
92	True	(not checked)	(not checked)	A-1 Grade
75	False	True	(not checked)	A Grade
60	False	False	True	B Grade
41	False	False	False	Fail or below B

Loan approval with nested if.

```
age = 20
salary = 35000
if age ≥ 18:
    # passed first check
    if salary ≥ 30000:
        # passed second check
        print("Loan APPROVED!")
    else:
        print("Salary too low for loan.")
else:
    print("Too young for loan.")
```

OUTPUT

```
Loan APPROVED!
```

age	salary	Outer check (age ≥ 18)	Inner check (salary ≥ 30000)	Output
20	35000	True	True	Loan APPROVED!
20	25000	True	False	Salary too low for loan.
16	35000	False	(never checked)	Too young for loan.

When you only need a yes/no answer, two levels can often be joined with `and` — but then you lose the separate messages:

```
age = 20
salary = 25000
if age ≥ 18 and salary ≥ 30000:
    print("Loan APPROVED!")
else:
    print("Loan not approved.")
```

OUTPUT

Loan not approved.

REMEMBER IT

In an `if-elif-else` ladder, put the **strictest condition first** (highest marks first). Think of a ladder: you step off at the first rung that fits.

COMMON MISTAKE

Wrong order gives wrong answers. Here 85 marks gets "B Grade", because $85 \geq 60$ is True first and the other conditions are ignored:

```
marks = 85
if marks ≥ 60:
    print("B Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 80:
    print("A-1 Grade")
```

OUTPUT

B Grade

This is a **logical error**: no crash, just a wrong result. Fix it by checking ≥ 80 first.

اردو میں سمجھیں

`if-elif-else` میں کئی شرطیں اوپر سے نیچے ترتیب سے دیکھی جاتی ہیں، اور جو شرط سب سے پہلے درست ہو اسی کا بلاک چلتا ہے، باقی سب چھوڑ دی جاتی ہیں۔ نیسٹڈ ایف (Nested if) میں ایک `if` دوسرے `if` کے اندر ہوتا ہے، اور اندرونی شرط تب ہی دیکھی جاتی ہے جب باہرونی شرط درست ہو۔

EXAM TIP

Grade-calculator programs (`if-elif-else`) and "largest of three numbers" are very common long questions. Check conditions in the right order, indent nested blocks carefully (8 spaces for the inner block), and give a sample run. Tracing tables like the ones above earn full marks in "predict the output" questions.

CHECK YOURSELF

1. Predict the output:

```
temp = 30
if temp > 35:
    print("Hot")
elif temp > 25:
    print("Warm")
elif temp > 15:
    print("Pleasant")
else:
    print("Cold")
```

2. Predict the output:

```
x = 15
if x > 10:
    if x > 20:
        print("A")
    else:
        print("B")
else:
    print("C")
```

Answers: 1. Warm (30 > 35 is False; 30 > 25 is True, so the rest are skipped even though 30 > 15 is also True). 2. B (outer check 15 > 10 passes; inner check 15 > 20 fails).

Application and Impacts of Computing

This chapter shows you how computing technologies such as Artificial Intelligence, the Internet of Things and Data Analytics are changing life in Pakistan — in cities, hospitals, farms and classrooms. You will also learn how to judge information sources, weigh the good and bad effects of computing, and see how assistive technologies give everyone an equal chance.

اس باب میں آپ سیکھیں گے کہ مصنوعی ذہانت (AI)، انٹرنیٹ آف تھنگز (IoT) اور ڈیٹا اینالٹکس ہماری روزمرہ زندگی کو کیسے بدل رہے ہیں۔ آپ معلومات کے ذرائع کو پرکھنا، کمپیوٹنگ کے مثبت اور منفی اثرات سمجھنا، اور معذور افراد کی مدد کرنے والی ٹیکنالوجی کے بارے میں بھی جانیں گے۔

What you will learn

- Explain what Artificial Intelligence (AI) is and give everyday examples of it.
- Describe how IoT, Data Analytics and AI work together in a smart system.
- Name and explain the six components of an IoT system, with examples of each.
- List the five components of Data Analytics in the correct order.
- Compare IoT, AI and Data Analytics by definition, purpose, type of work and output.
- Describe uses of IoT in smart cities, industry, healthcare, education, homes and farming.
- Discuss the uses, benefits and challenges of AI in education in Pakistan.
- Classify information sources as primary, secondary or tertiary, and judge when a source is unreliable.
- Analyse positive and negative impacts of computing in five fields, and practise responsible use.
- Explain the four categories of assistive technologies, their importance, and related careers.

5.1 Introduction to Computing Impacts

LEARN IT

Computing means using computers and computer-based devices to store, process and share information. Today computing is an important part of your daily life. You use computers, mobile phones, the internet and smart devices almost everywhere — at home, in college and on the road.

In Pakistan, computing is becoming **vital** (very necessary) in:

- **Schools and colleges** — online admissions, e-books, online results of the Sindh boards.
- **Hospitals** — computerised patient records, digital X-rays, online appointment systems.
- **Banks** — ATMs, mobile banking apps, Raast instant payments, JazzCash and Easypaisa wallets.
- **Farms** — weather forecasts on mobile phones, soil sensors, tube-well control.
- **Industries** — automatic machines in textile mills in Karachi, Hyderabad and Faisalabad.

An **impact** is the effect something has on people and society. Computing has **positive impacts** (speed, convenience, new jobs) and **negative impacts** (privacy risks, addiction, job loss). In this chapter you will study three key technologies first — AI, IoT and Data Analytics — and then their uses and impacts.

WORKED EXAMPLE

A day in Sara's life in Karachi — spot the computing:

Step 1. 7:00 am — Her phone unlocks by recognising her face (AI).

Step 2. 7:30 am — She books a ride on a ride-hailing app; GPS shows the car's location (internet + location data).

Step 3. 9:00 am — College attendance is marked by a biometric (fingerprint) machine (IoT).

Step 4. 2:00 pm — She pays her fee through a mobile banking app (computing in business).

Step 5. 8:00 pm — YouTube suggests a physics lecture based on what she watched yesterday (AI).

Conclusion: in a single day, Sara uses computing in at least five different fields without even thinking about it.

اردو میں سمجھیں

کمپیوٹنگ آج ہماری روزمرہ زندگی کا اہم حصہ بن چکی ہے۔ پاکستان میں اسکول، اسپتال، بینک، کھیت اور صنعتیں سب کمپیوٹر اور انٹرنیٹ پر انحصار کرنے لگے ہیں۔ کمپیوٹنگ کے فائدے بھی ہیں اور نقصانات بھی، جنہیں ہم اس باب میں سمجھیں گے۔

CHECK YOURSELF

1. Name four places in Pakistan where computing has become vital.

2. What is meant by the "impact" of computing?

Answers: 1. Any four of: schools, hospitals, banks, farms, industries. 2. The effect computing has on people and society — it can be positive or negative.

5.1.1 Artificial Intelligence (AI) ★ Golden

LEARN IT

Artificial Intelligence (AI) is the ability of a computer system to perform tasks that normally need human thinking, such as recognising faces, understanding language and making decisions.

Understand the name in two parts:

- **Artificial** — it is *created by humans*; it is not natural.
- **Intelligence** — it can perform tasks that *require thinking*.

AI systems **use data to make decisions**. The more good data an AI system sees, the better it becomes at recognising patterns (repeating features) and predicting what to do.

Examples from the source:

1. **Face recognition** on a mobile phone to unlock the screen.
2. **YouTube suggesting videos** based on your interests.
3. **Chatbots** answering questions on a website (for example, a bank's help chat).

More Pakistani examples: Google Maps predicting traffic on Shahrah-e-Faisal, Urdu voice typing on your phone, banks flagging a suspicious card transaction, and AI tools that translate English into Urdu.

AI in smart systems

AI rarely works alone. It often acts as the **"brain"** of a system, working together with the **Internet of Things (IoT)** and **Data Analytics**:

- **IoT devices gather data** — they are the "eyes" (sensors).
- **Analytics sorts the data** — it organises and studies it.
- **AI makes the final decision** — it is the "brain".
- An **action** follows — something happens in the real world.

IoT (Eyes) — collects data → Analytics — sorts data → AI (Brain) — decides → Action

WORKED EXAMPLE

Smart air-conditioner in a hot Hyderabad room — trace the system:

Step 1 (IoT – eyes). A temperature sensor reads the room temperature: 36 °C.

Step 2 (Analytics). The readings of the last 10 minutes are organised: 34, 35, 35, 36 °C — the room is getting hotter.

Step 3 (AI – brain). The AI compares this with the comfortable limit (say 26 °C) and with the fact that a person is in the room (motion sensor). Decision: "Turn the AC on."

Step 4 (Action). A switch turns the AC on automatically.

No human pressed a button — the system sensed, thought and acted.

REMEMBER IT

Eyes – Sort – Brain – Do. IoT is the *eyes*, Analytics *sorts*, AI is the *brain*, and the actuator *does* the action. Think of a cricket fielder: his eyes see the ball (IoT), he judges speed and direction (analytics), decides to dive (AI), and his body moves (action).

COMMON MISTAKE

Mistake: "AI means robots." **Fix:** Most AI is just *software* — YouTube suggestions, spam filters and face unlock have no robot body. A robot may use AI, but AI itself is the thinking software.

اردو میں سمجھیں

مصنوعی ذہانت (AI) کو "مصنوعی" اس لیے کہتے ہیں کہ اسے انسانوں نے بنایا ہے، اور "ذہانت" اس لیے کہ یہ ایسے کام کر سکتی ہے جن کے لیے سوچنا پڑتا ہے۔ AI ڈیٹا کی مدد سے فیصلے کرتی ہے، جیسے چہرہ پہچان کر فون کھولنا یا یوٹیوب پر ویڈیوز تجویز کرنا۔ ایک اسمارٹ سسٹم میں IoT آنکھوں کا کام کرتا ہے اور AI دماغ کا۔

EXAM TIP

Common questions: "Define AI and give two examples" (2–3 marks), and "Explain how AI, IoT and Data Analytics work together" (4–5 marks).

To score full marks: (1) write the two-part meaning of the name — *artificial* = *made by humans*, *intelligence* = *can think/decide*; (2) say "AI uses data to make decisions"; (3) give at least two examples; (4) for the "together" question, draw the flow IoT → Analytics → AI → Action and explain it with the room-AC example.

CHECK YOURSELF

1. Why is AI called "artificial"?
2. In a smart system, which technology is the "brain" and which is the "eyes"?
3. Is a spam filter in Gmail an example of AI? Why?

Answers: 1. Because it is created by humans. 2. AI is the brain; IoT is the eyes. 3. Yes — it uses data (past spam emails) to decide whether a new email is spam, a task that needs "thinking".

5.1.2 Internet of Things (IoT) & Components ★ Golden

LEARN IT

The **Internet of Things (IoT)** is a system in which **physical devices are connected to the Internet** and can **collect and share data** to perform tasks **automatically**.

The "things" are ordinary objects — watches, cameras, cars, fans, water pumps — that have been given sensors and an internet connection so they can "talk" to each other and to us.

Source example: A **smartwatch** collects your heart-rate data and sends it to your mobile phone, where it is displayed.

Components of IoT systems

IoT relies on **six foundational components** to work properly.

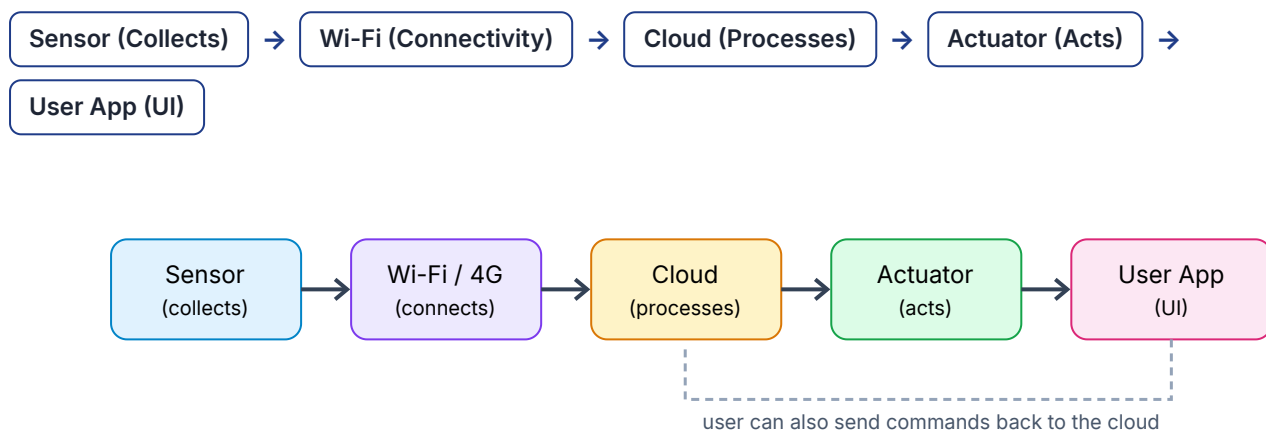
#	Component	What it does	Examples
1	Sensors	Devices that collect information from the physical world. They are the "eyes and ears" of the system.	Temperature sensor, motion sensor, fingerprint sensor, smoke sensor, soil moisture sensor
2	Actuators	Devices that perform physical actions in response to system commands. They convert digital signals into physical movement.	Electric motor, switch, automatic door opener, water valve, fan controller
3	Connectivity	The connection that allows devices to send and receive data . Without it, sensors can't talk to the system.	Wi-Fi, Bluetooth, ZigBee, 4G/5G
4	Data Processing	Changing raw data into useful information. Steps: collecting, organising, analysing and deciding.	Checking if the room temperature is too hot to trigger the AC
5	Cloud Computing	Storing data and using services over the internet instead of local hard drives. Handles massive IoT data.	Google Drive, iCloud, online LMS
6	User Interface (UI)	The component that enables interaction between the user and the device . Helps users see information and give commands.	Mobile apps, touch screens, voice commands

Understanding each component

- **Sensors** turn something in the real world (heat, movement, a fingerprint, smoke, wetness of soil) into data. A smoke sensor in a Karachi shopping mall "smells" smoke and reports it.

- **Actuators** do the opposite: they take a digital command and make something *move* or *change*. The water valve that opens, the motor that turns, the door that slides open at a bank entrance.
- **Connectivity** is the "road" for data. Short distances use Bluetooth (watch to phone); a house uses Wi-Fi; low-power sensor networks use **ZigBee**; devices far away in fields use mobile networks (4G/5G from Jazz, Zong, Telenor/Ufone).
- **Data processing** is where raw numbers ("36") become meaning ("too hot — turn on AC"). Its four steps are *collect* → *organise* → *analyse* → *decide*.
- **Cloud computing** stores and processes the huge amount of data that thousands of sensors produce every second — far more than a single small device can hold.
- **User Interface** is how *you* see the data and give orders — an app on your phone, a touch screen on a smart fridge, or saying "Hey Google, turn off the lights".

How the components connect



The six IoT components working as one system (data processing happens inside the cloud).

WORKED EXAMPLE

Smart fire alarm in a Karachi factory — identify all six components:

Step 1 — Sensor: A smoke sensor detects smoke near the machines.

Step 2 — Connectivity: It sends the reading over the factory Wi-Fi.

Step 3 — Cloud computing: The reading is stored on an online server along with the history of all sensors.

Step 4 — Data processing: The software checks: "Is the smoke level above the safe limit?" Yes → decide "Fire risk!"

Step 5 — Actuator: Water valves of the sprinklers open and the alarm bell switch turns on.

Step 6 — User interface: The manager's mobile app shows a red alert: "Smoke detected – Hall 3".

REMEMBER IT

Sensor senses, Actuator acts. Both words give you the answer: **sensor** = see/sense (input), **actuator** = **act** (output). Remember the six components with "**Some Ants Carry Data Carefully Up**" — Sensors, Actuators, Connectivity, Data processing, Cloud, UI.

COMMON MISTAKE

Mistake: Calling a fan controller or water valve a "sensor". **Fix:** Ask "does it *measure* something or *do* something?" A valve opens and closes — it *does*, so it is an **actuator**. A soil moisture sensor only *measures*, so it is a **sensor**.

اردو میں سمجھیں

انٹرنیٹ آف تھنگز (IoT) ایک ایسا نظام ہے جس میں مختلف آلات (جیسے گھڑیاں، کیمرے، گاڑیاں) انٹرنیٹ سے جڑے ہوتے ہیں اور خود کار طریقے سے ڈیٹا جمع اور شیئر کرتے ہیں۔ اس کے چھ بنیادی اجزاء ہیں: سینسرز (معلومات جمع کرتے ہیں)، ایکچوئٹرز (جسمانی کام انجام دیتے ہیں جیسے موٹر چلانا یا دروازہ کھولنا)، کنیکٹیویٹی (وائی فائی، بلوٹوتھ)، ڈیٹا پروسیسنگ، کلاؤڈ کمپیوٹنگ، اور یوزر انٹرفیس (موبائل ایپ)۔

EXAM TIP

"Define IoT and explain its components" is one of the most frequent long questions. Write the definition with the three key ideas — *physical devices, connected to the internet, collect and share data automatically*. Then list all **six** components, each with a one-line meaning and one example. A labelled flow diagram (Sensor → Wi-Fi → Cloud → Actuator → App) earns extra credit.

Short questions often ask: "Differentiate between a sensor and an actuator" — sensor collects data (input, eyes and ears), actuator performs physical action (output, converts digital signals into movement).

CHECK YOURSELF

1. Classify as sensor or actuator: (a) fingerprint scanner, (b) automatic door opener, (c) motion detector, (d) electric motor.
2. Name four types of connectivity used in IoT.
3. Why is cloud computing needed in IoT?

Answers: 1. (a) sensor, (b) actuator, (c) sensor, (d) actuator. 2. Wi-Fi, Bluetooth, ZigBee, 4G/5G. 3. Because IoT devices produce massive amounts of data; the cloud stores and processes it over the internet instead of on small local hard drives.

5.1.3 Data Analytics & Components

LEARN IT

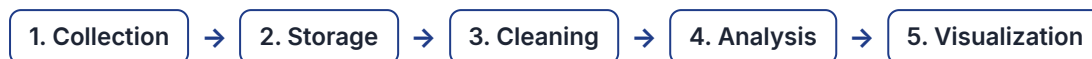
Data Analytics is the process of **collecting, organising, cleaning, studying and presenting data** to find useful information. It helps people make better decisions.

Data means raw facts and figures (for example, "72, 45, 88"). **Information** is data that has been processed so it has meaning ("Ali scored 72%; he passed"). Data Analytics is the bridge from data to information.

Source example: A college collects exam marks, calculates percentages, and identifies which students need extra help. This entire process is Data Analytics.

Components of Data Analytics

Data Analytics comprises **five key components**, which happen in order like a pipeline:



- 1. Data Collection** — Gathering raw information from multiple sources: websites, surveys, sensors, hospital records, bank transactions.
- 2. Data Storage** — Storing data safely and properly after collection. It can be stored in files, databases, spreadsheets or the cloud.
- 3. Data Cleaning** — Identifying and correcting errors in data. Raw data may contain:
 - **Duplication** of records (the same student entered twice),
 - **Incorrect values** (typos, e.g. marks "870" instead of "87"),
 - **Missing** files or records (a student's marks not entered),
 - **Improper formats** (dates written as 05/10/2026 in one place and "5 Oct 26" in another).
- 4. Data Analysis** — Studying the clean data to identify patterns, trends and relevant information for problem-solving (e.g. finding out which subjects students failed the most).
- 5. Data Visualization** — Creating visual representations of data (plots, charts, graphs) to provide valuable insights at a glance (e.g. a pie chart showing pass vs fail percentage).

WORKED EXAMPLE

A college in Sukkur analyses Physics test results (out of 50) for 6 students.

Step 1 — Collection: Marks are gathered from the teacher's register.

Roll No	Name	Marks (raw)
1	Ayesha	42
2	Bilal	18
3	Hina	35
3	Hina	35
4	Kashif	250
5	Sana	(missing)

Step 2 — Storage: The marks are typed into a spreadsheet saved on Google Drive (cloud).

Step 3 — Cleaning: Hina's record is a *duplicate* → delete one copy. Kashif's 250 is an *incorrect value* (more than 50) → the register shows 25, so correct it. Sana's marks are *missing* → ask the teacher; she was absent, so mark her "absent".

Step 4 — Analysis: Clean marks: 42, 18, 35, 25. Pass mark is 17 (33% of 50 = 16.5, rounded up). All 4 passed. Percentages: Ayesha 84%, Bilal 36%, Hina 70%, Kashif 50%. Average = $(42 + 18 + 35 + 25) \div 4 = 120 \div 4 = 30$ marks = 60%. Bilal is closest to failing → needs extra help.

Step 5 — Visualization: A bar chart shows each student's percentage; Bilal's short bar makes the problem visible at a glance.

REMEMBER IT

"Cows Sit Calmly And Vote" — Collection, Storage, Cleaning, Analysis, Visualization. Or think of cooking daal: buy the lentils (collect), keep them in a jar (store), pick out stones (clean), cook (analyse), serve beautifully on a plate (visualise).

COMMON MISTAKE

Mistake: Skipping cleaning and going straight to analysis. **Fix:** Dirty data gives wrong answers. In the example, Kashif's wrong "250" would have made the average $(42 + 18 + 35 + 250) \div 4 = 86.25$ — more than 50, which is impossible. Always clean first.

اردو میں سمجھیں

ڈیٹا اینالٹکس ڈیٹا کو جمع کرنے، ترتیب دینے، صاف کرنے، اس کا مطالعہ کرنے اور پیش کرنے کا عمل ہے تاکہ مفید معلومات حاصل ہوں اور بہتر فیصلے کیے جاسکیں۔ اس کے پانچ اجزاء ہیں: ڈیٹا جمع کرنا، محفوظ کرنا، صاف کرنا (غلطیاں اور دہرائے گئے ریکارڈ ہٹانا)، تجزیہ کرنا، اور چارٹ یا گراف کی صورت میں دکھانا۔

CHECK YOURSELF

1. List the five components of Data Analytics in order.
2. Name four kinds of errors removed during data cleaning.
3. A pie chart of pass/fail percentage belongs to which component?

Answers: 1. Collection, Storage, Cleaning, Analysis, Visualization. 2. Duplicate records, incorrect values (typos), missing records, improper formats. 3. Data Visualization.

Comparison: IoT vs AI vs Data Analytics

LEARN IT

These three technologies are often used together, so students mix them up. The table below, from the source, compares them on four aspects.

Aspect	IoT	Artificial Intelligence (AI)	Data Analytics
Definition	Network of physical devices connected to the internet.	Technology enabling machines to think and decide.	Process of examining data to find patterns.
Main purpose	Data collection and automation.	Intelligent decision making and predictions.	Extracting meaningful insights.
Type of work	Works with physical devices (hardware).	Works with smart software systems.	Works with data (numbers, records).
Output	Raw data from the environment.	Smart decisions or actions.	Reports, graphs, insights.
Role in a smart system	Eyes and hands	Brain	Organiser / reporter
Pakistani example	Safe City cameras in Lahore/Karachi capturing traffic	Software recognising a car's number plate for an e-challan	Monthly report showing which roads have the most violations

WORKED EXAMPLE

One problem, three technologies — a K-Electric smart meter:

IoT: The smart electricity meter measures units used every 15 minutes and sends them over the mobile network. Output: raw readings.

Data Analytics: The company cleans and studies a month of readings and draws a graph showing that most electricity is used between 7 pm and 11 pm. Output: report/graph.

AI: Software learns each area's pattern, predicts tomorrow's demand, and decides how to plan load management. Output: smart decision.

REMEMBER IT

Ask "What is the output?" Raw readings → IoT. Charts and reports → Data Analytics. A decision or prediction → AI.

COMMON MISTAKE

Mistake: Writing that IoT "makes decisions". **Fix:** In the comparison table IoT's main purpose is *data collection and automation*; the *intelligent decision* belongs to AI.

اردو میں سمجھیں

IoT جسمانی آلات (ہارڈ ویئر) کا نیٹ ورک ہے جو ماحول سے خام ڈیٹا جمع کرتا ہے۔ AI ذہین سافٹ ویئر ہے جو فیصلے اور پیش گوئیاں کرتا ہے۔ ڈیٹا اینالیٹکس ڈیٹا کا جائزہ لے کر رپورٹس اور گراف بناتا ہے۔ تینوں مل کر ایک اسمارٹ نظام بناتے ہیں۔

CHECK YOURSELF

1. Which of the three works mainly with hardware?
2. What is the output of Data Analytics?

Answers: 1. IoT. 2. Reports, graphs and insights.

5.2 Uses of IoT in Different Areas

LEARN IT

IoT **improves efficiency** (doing more with less effort), **saves time**, and **enhances the overall quality of life**. It plays a significant role in developing modern **infrastructure** (the basic systems of a country — roads, power, water, hospitals). The source lists six areas.

1. Smart Cities

- **Traffic management:** Sensors adjust traffic signals automatically to reduce congestion (crowding). Example: signals on a busy Karachi road stay green longer for the direction with more cars.
- **Street lighting:** Lights turn ON/OFF based on daylight or movement, saving electricity.
- **Parking:** Sensors guide drivers to empty parking spaces, e.g. in a large shopping mall car park.
- **Waste management:** Smart bins send alerts when full, so trucks collect only full bins instead of driving to every bin.

2. Industrial Automation

- Sensors check machine **temperature and vibration** to prevent breakdowns (called predictive maintenance).
- Automated machines perform repeated tasks precisely.
- Quality-control sensors check products during manufacturing — for example, in the **textile industries of Sindh**, cameras and sensors spot faults in cloth as it is woven.

3. Healthcare (Smart Health)

- **Smartwatches** continuously measure heart rate and vital signs (pulse, oxygen level).
- **Remote monitoring devices** send patient data directly to doctors — vital for **rural areas of Pakistan**, where a specialist may be hundreds of kilometres away (e.g. a heart patient in Tharparkar monitored by a doctor in Karachi).
- **Emergency alert devices** inform hospitals instantly, for example when an elderly person falls.

4. Education (Smart Learning)

- **Smart classrooms:** digital boards and connected projectors.
- **Attendance systems:** biometric automated attendance for students and teachers (used in many Sindh government schools and colleges).
- **Resource management:** automated classroom lighting and AC control — off when the room is empty.

5. Home Automation

- **Smart lighting:** controlled remotely via mobile apps.
- **Security:** real-time monitoring via Wi-Fi cameras — you can watch your house gate on your phone.
- **Voice control:** using assistants (like Alexa or Google Assistant) to control TVs and appliances.

6. Smart Farming

Smart farming is **crucial for agriculture in Pakistan** (especially Sindh and Punjab) to **save water** and **improve yields** (the amount of crop produced).

- **Soil moisture sensors:** check if crops actually need water.
- **Automated irrigation:** turns water pumps ON/OFF based on soil data.

- **Drones:** monitor crop health from above, spotting disease or dry patches.

Smart farming process:



WORKED EXAMPLE

A cotton farmer near Nawabshah uses smart irrigation:

Step 1. A soil moisture sensor in the field reads 18% moisture.

Step 2. It sends the reading over the 4G network to a cloud service.

Step 3. The cloud AI checks: cotton needs at least 25% moisture, and the weather forecast shows no rain → decision "Needs water: YES".

Step 4. The actuator (pump switch) turns the tube-well ON.

Step 5. When moisture reaches 30%, the AI decides "enough" and the pump turns OFF.

Result: water is used only when needed — important because Sindh faces water shortages — and the farmer sees everything on his phone app.

REMEMBER IT

Remember the six areas with "**CHIEF H**": **C**ities, **H**ealth, **I**ndustry, **E**ducation, **F**arming, **H**omes.

COMMON MISTAKE

Mistake: Giving only the name of the area ("smart cities") in an exam answer. **Fix:** Always add *what* IoT does there — e.g. "smart bins send alerts when full" — and, if possible, a Pakistani example.

اردو میں سمجھیں

IoT کارکردگی بہتر بناتا ہے، وقت بچاتا ہے اور زندگی کا معیار بلند کرتا ہے۔ اسمارٹ شہروں میں ٹریفک سگنل اور اسٹریٹ لائٹس خود کار ہوتی ہیں، اسپتالوں میں مریضوں کا ڈیٹا براہ راست ڈاکٹر تک پہنچتا ہے، اور کھیتوں میں مٹی کی نمی کے سینسر بتاتے ہیں کہ فصل کو پانی چاہیے یا نہیں، جس سے سندھ اور پنجاب میں پانی کی بچت ہوتی ہے۔

CHECK YOURSELF

1. How do smart bins help waste management?
2. Why is remote health monitoring especially useful in Pakistan?
3. In smart farming, which device is the actuator?

Answers: 1. They send alerts when full, so only full bins are emptied. 2. Rural areas lack doctors; devices send patient data directly to doctors far away. 3. The water pump (its switch).

5.2.1 Scope of AI in Education in Pakistan

LEARN IT

Artificial Intelligence is gradually becoming an important part of Pakistan's education system, improving **teaching, learning** and **administration** (managing the school: admissions, records, timetables).

Practical uses:

- **Learning Management Systems (LMS)** — online platforms (e.g. Google Classroom, Moodle) where teachers post lessons and assignments; AI features can recommend lessons to each student.
- **Automated grading and digital testing** — MCQ tests marked instantly by computer.
- **AI-based career counselling** — software that studies a student's marks and interests and suggests suitable fields.

Benefits:

- Helps students learn **at their own speed**.
- **Identifies students who need extra help**.
- Supports students with disabilities (**Assistive AI**), e.g. text-to-speech for a visually impaired student.

Major challenges in Pakistan:

- Limited **internet access in rural areas**.
- Lack of **computers and digital devices** for all students.
- Need for proper **teacher training** to use AI tools.
- **Data privacy and security** issues — students' personal data must be protected.

Practical uses	Benefits	Challenges
LMS	Learn at own speed	Limited rural internet
Automated grading & digital testing	Identifies students needing help	Lack of computers/devices
AI career counselling	Supports students with disabilities	Teacher training needed
—	—	Data privacy & security

WORKED EXAMPLE

Scenario — Fatima in Larkana and an AI tutor app:

Step 1. Fatima takes a 10-question online maths quiz on the college LMS. It is graded instantly (automated grading).

Step 2. The AI notices she got all 3 trigonometry questions wrong.

Step 3. It recommends two short videos and five easy practice questions on trigonometry — she learns at her own speed.

Step 4. Her teacher's dashboard flags "Fatima – needs help in trigonometry" (identifies students who need help).

Challenge: At home Fatima's village has weak 3G signals, so the videos buffer — showing the rural internet challenge.

COMMON MISTAKE

Mistake: Writing that AI will replace teachers. **Fix:** The source says AI *improves* teaching and needs *teacher training* — teachers use AI as a tool; they are not replaced.

اردو میں سمجھیں
پاکستان کے تعلیمی نظام میں مصنوعی ذہانت آہستہ آہستہ اہم کردار ادا کر رہی ہے، جیسے لرننگ مینجمنٹ سسٹم (LMS)، خودکار جانچ اور کیریئر کونسلنگ۔ اس سے طلبہ اپنی رفتار سے سیکھ سکتے ہیں اور کمزور طلبہ کی نشاندہی ہوتی ہے۔ لیکن دیہی علاقوں میں انٹرنیٹ کی کمی، کمپیوٹرز کی قلت، اساتذہ کی تربیت اور ڈیٹا کی حفاظت بڑے چیلنج ہیں۔

CHECK YOURSELF

1. Give three practical uses of AI in education.
2. State any two challenges of AI in education in Pakistan.

Answers: 1. LMS, automated grading and digital testing, AI-based career counselling. 2. Any two: limited rural internet, lack of devices, need for teacher training, data privacy and security.

5.2.2 Career Connection in IoT and AI

LEARN IT

Learning computing skills opens doors to many employers in Pakistan: **software houses** (e.g. in Karachi, Lahore and Islamabad technology parks), **telecom companies** (Jazz, Telenor), **banks**, and **government IT departments** (e.g. NADRA, provincial IT boards).

- **IoT Engineer:** designs smart systems and connected devices (e.g. a smart irrigation kit).
- **Data Analyst:** analyses the data generated by IoT systems and produces reports.

Key skills needed:

- **Programming** — Python, C++.
- **Networking** — Wi-Fi, protocols (rules for how devices communicate).
- **Electronics** — sensors and circuits.
- **Problem-solving.**

REMEMBER IT

Skills = "**PNEP**": Programming, Networking, Electronics, Problem-solving. You start building the first one this year when you learn Python!

اردو میں سمجھیں
کمپیوٹنگ کی مہارتیں سیکھ کر آپ سافٹ ویئر ہاؤسز، ٹیلی کام کمپنیوں (جیسے جاز، ٹیلی نار)، بینکوں اور سرکاری آئی ٹی محکموں میں کام کر سکتے ہیں۔ IoT مینجمنٹ سمارٹ آلات بناتا ہے اور ڈیٹا اینالسٹ ان سے پیدا ہونے والے ڈیٹا کا تجزیہ کرتا ہے۔ اس کے لیے پروگرامنگ، نیٹ ورکنگ، الیکٹرانکس اور مسئلہ حل کرنے کی صلاحیت ضروری ہے۔

CHECK YOURSELF

1. What does an IoT Engineer do?
2. Name the four key skills for IoT/AI careers.

Answers: 1. Designs smart systems and connected devices. 2. Programming (Python, C++), networking, electronics, problem-solving.

5.3 Information Sources (Primary, Secondary, Tertiary) ★ Golden

LEARN IT

Information sources are ways to find knowledge and data. The internet has vast information, but **not all of it is accurate**. Sources are divided into three main categories.

1. Primary sources give **original, firsthand information** straight from people, events or official records. They are **highly authentic** (genuine).

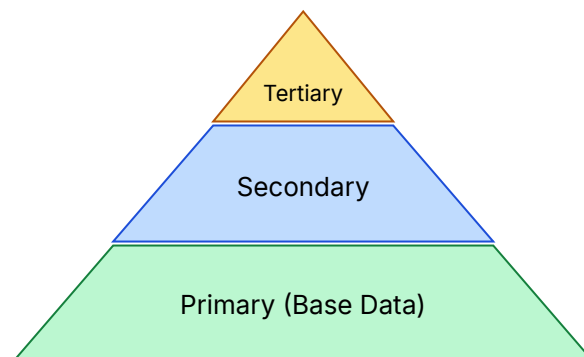
Examples: interviews, photographs, diaries, NADRA official records.

2. Secondary sources contain information that has **already been collected and explained by someone else**. They **summarise or analyse primary data**.

Examples: textbooks, news reports, research papers, Pakistan Bureau of Statistics reports.

3. Tertiary sources provide **highly summarised and organised information** derived from both primary and secondary sources. They are used for **quick reference**.

Examples: encyclopedias, Wikipedia, dictionaries, handbooks.



The information pyramid: primary data is the base; each higher level is built from the levels below it and is more summarised.

Many examples, sorted

Primary (firsthand)	Secondary (explained by others)	Tertiary (summarised, quick reference)
Interview with a flood victim in Dadu	Newspaper report on the 2022 floods	Encyclopedia entry "Floods in Pakistan"
Photograph taken at an event	Textbook chapter explaining the event	Wikipedia article
Personal diary or letter	Biography written by a historian	Dictionary
NADRA official records (CNIC data)	Pakistan Bureau of Statistics report analysing census data	Handbook / almanac of facts
Original survey responses	Research paper analysing the survey	Index or list of research papers
Sensor readings from an IoT weather station	TV news analysis of the weather	A school "fact sheet" on climate
Quaid-e-Azam's original speech recording	Article discussing the speech	Encyclopedia summary of his life

When do sources become unreliable?

In the digital age you must check facts carefully. A source is **unreliable** (cannot be trusted) if:

Source type	Reasons for unreliability
Primary	Information is incomplete, data is recorded incorrectly, or false information was deliberately provided.
Secondary	The information is outdated, lacks proper references/citations, or has not been verified by an expert.
Tertiary	Comes from an unknown/unverified website, or the information is oversimplified and misses key facts.

REMEMBER IT

Rule of thumb (Author – Date – References): If an online article does not state the **author's name**, the **date of publication**, or list its **references**, it should not be trusted for academic work. Remember **"ADR"** — like checking a bus has a driver, a route and a ticket before you board.

WORKED EXAMPLE

Classify these — worked solutions:

(a) **Your grandfather's handwritten diary from 1947.** Written by a person who lived through the event → firsthand → **Primary**.

(b) **A Dawn newspaper article analysing the budget.** A journalist collected facts and explained them → **Secondary**.

(c) **An Oxford English–Urdu dictionary.** Organised, summarised reference built from many sources → **Tertiary**.

(d) **A Bureau of Statistics report on literacy in Sindh.** It analyses census (primary) data → **Secondary**.

(e) **A photo you took of a road accident.** Original record of the event → **Primary**.

Method: Ask (1) "Was the creator *there* / is it the original record?" → Primary. (2) "Is someone *explaining or analysing* original data?" → Secondary. (3) "Is it a *quick-reference summary* of many sources?" → Tertiary.

CHECK YOURSELF

Exercise — classify these as Primary (P), Secondary (S) or Tertiary (T):

1. A video interview with a cricketer after a match.
2. Your Computer Science textbook.
3. Wikipedia page on "Mohenjo-daro".
4. A birth certificate issued by the union council.
5. A research paper reviewing ten studies on smog in Lahore.
6. A handbook of physics formulas.
7. Original exam answer sheets of students.
8. A TV news report summarising a court decision.

Answers: 1. P 2. S 3. T 4. P 5. S 6. T 7. P 8. S

COMMON MISTAKE

Mistake: Thinking "primary" means "most important" or "tertiary" means "useless". **Fix:** The names describe *how far the information is from the original event*. Primary = straight from the source; tertiary = summarised summary. Each is useful for a different job. Also, Wikipedia is **tertiary** — not primary — because it summarises other sources.

اردو میں سمجھیں

معلومات کے ذرائع علم اور ڈیٹا تلاش کرنے کے طریقے ہیں، جنہیں تین اقسام میں تقسیم کیا جاتا ہے۔ پرائمری ذرائع براہ راست اور اصل معلومات دیتے ہیں (جیسے انٹرویو، تصاویر، نادر اکا ریکارڈ)۔ سیکنڈری ذرائع وہ ہیں جن میں کسی اور نے پرائمری معلومات کی وضاحت یا تجزیہ کیا ہو (جیسے درسی کتابیں، خبریں)۔ ٹرشیئر ذرائع خلاصہ شدہ معلومات دیتے ہیں جیسے انسائیکلو پیڈیا، ویکی پیڈیا اور لغت۔ جس آن لائن مضمون میں مصنف کا نام، تاریخ یا حوالہ جات نہ ہوں اس پر بھروسہ نہ کریں۔

EXAM TIP

Typical questions: "Differentiate between primary, secondary and tertiary sources with two examples each" (long/short), "Classify the following sources..." (MCQ/short), and "When does a source become unreliable?"

Write the definition key phrase for each — *firsthand/original, collected and explained by someone else, highly summarised for quick reference* — plus two examples each. Draw the pyramid with primary at the base. For unreliability, give one reason for each type and quote the author–date–references rule.

5.4 Impacts of Computing in Various Fields

LEARN IT

Computing has drastically changed society. It makes life **faster and more connected**, but it also brings challenges like **privacy risks** and **technology addiction**. For every field, think of both sides.

Field	Positive impact (+)	Negative impact (–)
Education	Online classes, e-books and learning apps make studying highly flexible and accessible.	Excessive screen time reduces attention spans and causes physical health issues (eye strain, back pain).
Communication	Emails and video calls allow instant, global communication at almost zero cost (e.g. a WhatsApp video call to a relative in Dubai).	People interact less in person, leading to social isolation and reduced face-to-face skills.
Business	Online shopping, digital banking and automation dramatically improve speed and efficiency.	Traditional manual jobs are lost to machines; risk of severe cyber-attacks on banks.
Healthcare	Digital patient records and telemedicine allow doctors to treat remote patients accurately.	Patient data privacy is at risk if hospital servers are hacked.
Agriculture	Smart irrigation and weather forecasting massively increase crop yields.	Poor internet access in rural areas means many farmers cannot use these tools.

Smart and responsible use

- **Protect personal information** and use **strong passwords** (long, with letters, numbers and symbols; never share your OTP or CNIC details with callers).
- **Do not share fake news** or unverified WhatsApp forwards.
- **Balance screen time** with physical sports and activities.

WORKED EXAMPLE

Scenario analysis — a bank in Karachi moves to mobile banking:

Step 1 — Identify the field: Business (banking).

Step 2 — Positive: Customers transfer money in seconds from home; no queue at the branch; the bank processes more transactions with fewer staff (speed and efficiency).

Step 3 — Negative: Some cashier jobs are no longer needed (job loss); hackers may attempt cyber-attacks or phishing SMS to steal accounts.

Step 4 — Responsible use: Customers should use strong passwords, never share OTPs, and report suspicious messages.

REMEMBER IT

For the five fields, remember "**E-C-B-H-A**" — **E**ducation, **C**ommunication, **B**usiness, **H**ealthcare, **A**griculture — and always write one + and one – for each.

COMMON MISTAKE

Mistake: Writing only positive impacts in a "discuss the impacts" question. **Fix:** "Impact" means *both* sides. Examiners expect a balanced answer: advantages, disadvantages, and how to use technology responsibly.

اردو میں سمجھیں

کمپیوٹنگ نے معاشرے کو بہت بدل دیا ہے۔ تعلیم، مواصلات، کاروبار، صحت اور زراعت میں اس نے کام تیز اور آسان بنائے ہیں، لیکن اس کے ساتھ اسکرین کی لت، سماجی تنہائی، ملازمتوں کا خاتمہ، ہیکنگ اور پرائیویسی کے خطرات بھی پیدا ہوئے ہیں۔ ذمہ دارانہ استعمال کے لیے مضبوط پاس ورڈ رکھیں، جعلی خبریں نہ پھیلانیں، اور کھیل کود کے ساتھ اسکرین ٹائم میں توازن رکھیں۔

CHECK YOURSELF

1. Give one positive and one negative impact of computing in healthcare.
2. Why can't many farmers in Pakistan benefit from smart agriculture tools?
3. State three rules of smart and responsible use.

Answers: 1. + Digital records and telemedicine help treat remote patients; – patient data privacy is at risk if servers are hacked. 2. Poor internet access in rural areas. 3. Protect personal information/use strong passwords; do not share fake news; balance screen time with physical activity.

5.5 Assistive Technologies

★ Golden

LEARN IT

Assistive technologies are **devices, software and systems** that help people with **disabilities or functional limitations** (difficulty in seeing, hearing, moving, speaking, remembering, etc.). They allow more people to **participate equally** in school, work and society.

There are **four categories**:

Category	Purpose	Examples
1. Mobility aids	Help people move from one place to another.	Smart wheelchairs, stair lifts
2. Sensory aids	Support vision and hearing to improve interaction.	Hearing aids, screen readers (VoiceOver), Braille displays
3. Communication aids	Assist individuals with speech difficulties to express themselves.	Speech-generating devices, text-to-speech apps
4. Cognitive / learning aids	Help people with memory, reading and understanding.	Audiobooks, spell-checkers, digital reminder apps

Mobility · Sensory · Communication · Cognitive

- A **screen reader** is software that reads aloud the text on a screen, so a blind person can use a phone or computer. **VoiceOver** is Apple's screen reader; Android has TalkBack.
- A **Braille display** is a device with small pins that rise and fall to form Braille dots, so a blind user can read screen text with their fingers.
- A **speech-generating device** speaks out words that a person types or selects on a screen.
- A **smart wheelchair** can be controlled by a joystick, voice or head movement and may avoid obstacles with sensors.

WORKED EXAMPLE

Scenario — choose the right assistive technology:

(a) Ahmed, a student in Hyderabad, is blind and wants to read his e-textbook. → **Sensory aid**: a screen reader or Braille display.

(b) Mrs Rashida cannot climb the stairs of her house after a knee operation. → **Mobility aid**: stair lift (or smart wheelchair).

(c) Zain cannot speak clearly due to a medical condition but can type. → **Communication aid**: text-to-speech app or speech-generating device.

(d) Nani often forgets to take her medicine. → **Cognitive aid**: digital reminder app.

(e) Usman has dyslexia (difficulty reading). → **Cognitive/learning aid**: audiobooks and spell-checkers.

Method: identify the *limitation* (moving, seeing/hearing, speaking, memory/reading) → choose the matching category.

REMEMBER IT

Four categories = "**My Small Cat Cries**": **M**obility, **S**ensory, **C**ommunication, **C**ognitive. Link each to a body function: *legs* (mobility), *eyes/ears* (sensory), *mouth* (communication), *mind* (cognitive).

COMMON MISTAKE

Mistake: Putting text-to-speech and screen readers in the same category every time. **Fix**: In the source, the **screen reader** is a *sensory* aid (supports vision), while a **text-to-speech app** is listed as a *communication* aid (helps a person with speech difficulty express themselves). Decide by *whose problem* it solves.

معاون ٹیکنالوجیز (Assistive Technologies) وہ آلات، سافٹ ویئر اور نظام ہیں جو معذور افراد کی مدد کرتے ہیں تاکہ وہ اسکول، کام اور معاشرے میں برابر حصہ لے سکیں۔ ان کی چار اقسام ہیں: نقل و حرکت میں مدد (اسمارٹ وہیل چیئر)، حسی مدد (سماعتی آلہ، اسکرین ریڈر، بریل ڈسپلے)، گفتگو میں مدد (اسپیچ جنرینگ ڈیوائس) اور ذہنی و تعلیمی مدد (آڈیو بکس، اسپیل چیکر، یاد دہانی ایپس)۔

EXAM TIP

Expect: "What are assistive technologies? Explain their categories with examples" (long), "Name two sensory aids" (short), or "A screen reader is an example of ___" (MCQ).

For full marks write the definition (devices, software and systems; help people with disabilities; equal participation), then all **four** categories each with purpose + two examples. Finish with one line on importance (equal access, independence, inclusion).

CHECK YOURSELF

1. Which category does a hearing aid belong to?
2. Give two examples of cognitive/learning aids.
3. Define assistive technologies in one sentence.

Answers: 1. Sensory aids. 2. Any two: audiobooks, spell-checkers, digital reminder apps. 3. Devices, software and systems that help people with disabilities or functional limitations participate equally in school, work and society.

5.5.1 Importance of Assistive Technologies

LEARN IT

Assistive technologies are important for three main reasons:

1. **Equal access to education:** Screen readers allow blind students to read digital textbooks equally with their peers (classmates).
2. **Independence in daily life:** Mobility aids reduce the need for constant human assistance — a person in a smart wheelchair can go to the market without someone pushing.
3. **Social inclusion:** They break down barriers, allowing individuals to work, study and socialise confidently.

Equal Access → Independence → Inclusion

WORKED EXAMPLE

How one tool changes a life: A visually impaired student in Karachi uses a screen reader on her laptop. **Equal access** — she reads the same e-books as her class. **Independence** — she no longer needs someone to read to her. **Inclusion** — she joins the class WhatsApp group, takes online tests and later gets a job, contributing to society.

REMEMBER IT

Remember **"AII"**: Access, Independence, Inclusion.

معاون ٹیکنالوجی کی اہمیت تین باتوں میں ہے: تعلیم تک برابر رسائی (ناہینا طلبہ اسکرین ریڈر سے کتابیں پڑھ سکتے ہیں)، روزمرہ زندگی میں خود مختاری (کسی کی مسلسل مدد کی ضرورت کم ہوتی ہے)، اور سماجی شمولیت (لوگ اعتماد کے ساتھ پڑھ، کام اور میل جول کر سکتے ہیں)۔

CHECK YOURSELF

1. State three reasons why assistive technologies are important.
2. How do mobility aids support independence?

Answers: 1. Equal access to education, independence in daily life, social inclusion. 2. They reduce the need for constant human assistance.

5.5.2 Career Connection in Assistive Tech

LEARN IT

Designing assistive technology is a **highly rewarding career path** — you earn a living while improving people's lives.

Career	What they do
Assistive Software Developer	Creates tools such as speech-to-text software (e.g. Urdu voice typing for people who cannot use a keyboard).
Biomedical Engineer	Designs artificial limbs and hearing aids.
UX/UI Accessibility Expert	Ensures websites are easy to navigate for visually impaired users (e.g. adding text descriptions to images so screen readers can read them).

UX means *user experience* (how easy and pleasant a product is to use); **UI** means *user interface* (the screens and buttons you interact with). **Accessibility** means designing so that *everyone*, including people with disabilities, can use it.

معاون ٹیکنالوجی بنانا ایک قابل قدر پیشہ ہے۔ اسٹیو سافٹ ویئر ڈویلپر اسپیچ ٹو ٹیکسٹ جیسے ٹولز بناتا ہے، ہائیو میڈیکل انجینئر مصنوعی اعضاء اور سماعتی آلات تیار کرتا ہے، اور UX/UI ایسیسبیلٹی ماہر یقینی بناتا ہے کہ ویب سائٹس ناہینا افراد کے لیے بھی آسانی سے استعمال ہو سکیں۔

CHECK YOURSELF

1. Who designs artificial limbs and hearing aids?
2. What does a UX/UI accessibility expert ensure?

Answers: 1. A biomedical engineer. 2. That websites are easy to navigate for visually impaired users.

Chapter 5 Review

Chapter summary

- Computing is vital in Pakistan's schools, hospitals, banks, farms and industries; it has both positive and negative impacts.
- **AI** is "artificial" (made by humans) and "intelligent" (does tasks needing thinking); it uses data to learn, recognise patterns and make decisions.
- In smart systems, IoT is the "eyes" that gather data, Analytics sorts it, and AI is the "brain" that decides the action.
- **IoT** is a system of physical devices connected to the internet that collect and share data to perform tasks automatically.
- The six IoT components are sensors (collect), actuators (act), connectivity, data processing, cloud computing and user interface.
- **Data Analytics** is a pipeline: collection → storage → cleaning → analysis → visualization.
- Data cleaning removes duplicates, incorrect values, missing records and improper formats.
- IoT outputs raw data (hardware); AI outputs smart decisions (software); Data Analytics outputs reports, graphs and insights (data).
- IoT is used in smart cities, industrial automation, healthcare, education, home automation and smart farming.
- Smart farming (soil sensor → cloud AI → water pump) saves water and improves yields in Sindh and Punjab.
- AI in Pakistani education: LMS, automated grading, career counselling; challenges are rural internet, lack of devices, teacher training and data privacy.
- IoT/AI careers (IoT engineer, data analyst) need programming, networking, electronics and problem-solving.
- Information sources: primary (firsthand/original), secondary (collected and explained by others), tertiary (summarised for quick reference).
- Do not trust an online article that lacks an author, a publication date or references.
- Computing helps education, communication, business, healthcare and agriculture but risks screen addiction, isolation, job loss, cyber-attacks and privacy loss.
- **Assistive technologies** — mobility, sensory, communication and cognitive aids — give equal access, independence and social inclusion to people with disabilities.

Key terms

Term	Meaning	اردو
Computing	Using computers and digital devices to store, process and share information.	کمپیوٹنگ
Artificial Intelligence (AI)	Human-made technology that lets machines think, learn and decide using data.	مصنوعی ذہانت
Internet of Things (IoT)	Physical devices connected to the internet that collect and share data to work automatically.	انٹرنیٹ آف تھنگز، اشیاء کا انٹرنیٹ
Sensor	Device that collects information from the physical world.	حساس آلہ (سینسر)
Actuator	Device that performs a physical action in response to a command.	عمل کار آلہ (ایکچوٹر)
Connectivity	The link (Wi-Fi, Bluetooth, ZigBee, 4G/5G) that lets devices send and receive data.	رابطہ کاری
Data processing	Changing raw data into useful information.	ڈیٹا کی پروسیسنگ
Cloud computing	Storing data and using services over the internet instead of local drives.	کلاؤڈ کمپیوٹنگ
User interface (UI)	The part that lets a user see information and give commands to a device.	صارف انٹرفیس
Data Analytics	Collecting, storing, cleaning, analysing and presenting data to find useful information.	ڈیٹا کا تجزیہ
Data cleaning	Finding and correcting errors such as duplicates and missing values.	ڈیٹا کی صفائی
Data visualization	Showing data as charts, plots or graphs.	ڈیٹا کی بصری پیشکش
Smart city	City using IoT for traffic, lighting, parking and waste.	اسمارٹ شہر
Smart farming	Using sensors, automated irrigation and drones in agriculture.	اسمارٹ کاشتکاری
Learning Management System (LMS)	Online platform for lessons, assignments and tests.	آن لائن تعلیمی نظام
Telemedicine	Treating patients remotely using computing and communication.	دور دراز علاج
Primary source	Original, firsthand information.	بنیادی ذریعہ
Secondary source	Information collected and explained by someone else.	ثانوی ذریعہ
Tertiary source	Highly summarised information for quick reference.	ثالثی ذریعہ
Reliability	How far a source can be trusted.	قابل اعتماد ہونا
Privacy	Keeping personal information safe from others.	رازداری

Cyber-attack	An attempt by hackers to damage or steal from computer systems.	سائبر حملہ
Assistive technology	Devices, software and systems that help people with disabilities.	معاون ٹیکنالوجی
Screen reader	Software that reads screen text aloud for blind users.	اسکرین ریڈر
Braille display	Device with raised pins showing text as Braille dots.	بریل ڈسپلے
Accessibility	Designing so everyone, including people with disabilities, can use a product.	رسائی پذیری

Practice: multiple choice

- AI is called "artificial" because it:
 - runs only on electricity
 - is created by humans
 - is always wrong
 - works without data
- Which of the following is an actuator?
 - Temperature sensor
 - Motion sensor
 - Water valve
 - Wi-Fi router
- ZigBee is an example of which IoT component?
 - Connectivity
 - Sensor
 - User interface
 - Actuator
- Google Drive and iCloud are examples of:
 - Sensors
 - Data cleaning
 - Actuators
 - Cloud computing
- Removing duplicate student records belongs to which component of Data Analytics?
 - Data collection
 - Data storage
 - Data cleaning
 - Data visualization
- A pie chart showing pass vs fail percentage is an example of:
 - Data collection
 - Data storage
 - Data cleaning
 - Data visualization
- According to the comparison table, the output of IoT is:
 - Raw data from the environment
 - Smart decisions
 - Reports and graphs
 - Printed documents
- In a smart system, AI acts as the:
 - eyes
 - brain
 - hands
 - wires
- NADRA's official records are a:
 - tertiary source
 - secondary source
 - primary source
 - unreliable source

10. Wikipedia is classified as a:

- | | |
|---------------------|---------------------|
| a. primary source | b. tertiary source |
| c. secondary source | d. firsthand record |

11. Smart bins that send alerts when full are a use of IoT in:

- | | |
|--------------------|-----------------|
| a. smart farming | b. healthcare |
| c. home automation | d. smart cities |

12. A system in which a soil sensor tells the cloud AI to switch on a water pump is used in:

- | | |
|---------------------|--------------------------|
| a. smart farming | b. industrial automation |
| c. smart classrooms | d. traffic management |

13. A screen reader such as VoiceOver is a:

- | | |
|-----------------|----------------------|
| a. mobility aid | b. communication aid |
| c. sensory aid | d. cognitive aid |

14. A speech-generating device is a:

- | | |
|-----------------|----------------------|
| a. mobility aid | b. cognitive aid |
| c. sensory aid | d. communication aid |

15. An online article should NOT be trusted for academic work if it lacks:

- | | |
|-----------------------|--|
| a. colourful pictures | b. the author's name, publication date or references |
| c. a long title | d. advertisements |

Practice: short questions

1. Define Artificial Intelligence and give two examples from daily life.
2. Define the Internet of Things (IoT) with one example.
3. Differentiate between a sensor and an actuator with two examples of each.
4. What is the role of cloud computing in an IoT system?
5. List the five components of Data Analytics. What kinds of errors are removed in data cleaning?
6. Give two uses of IoT in smart cities and two in healthcare.
7. State two benefits and two challenges of AI in education in Pakistan.
8. Differentiate between primary and secondary sources with two examples each.
9. When does a secondary source become unreliable?
10. Give one positive and one negative impact of computing on (a) communication and (b) business.

Practice: long / problem questions

1. Define IoT. Explain its six components with an example of each. Draw a diagram showing how they connect.
2. Compare IoT, AI and Data Analytics in a table (definition, purpose, type of work, output). Then explain, with an example, how the three work together in a smart system.
3. A college in Mirpurkhas wants to find out why many students are failing Mathematics. Explain how it would use each of the five components of Data Analytics to solve this problem.

4. Explain the three types of information sources with examples. Classify the following: (i) a school textbook, (ii) a photograph of an event, (iii) an encyclopedia, (iv) an interview, (v) a Bureau of Statistics report, (vi) a dictionary. When does each type become unreliable?
5. What are assistive technologies? Explain their four categories with examples, state their importance, and name two related careers.
6. "Computing has both positive and negative impacts on society." Discuss with reference to education, healthcare and agriculture, and suggest three rules for smart and responsible use.

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. d | 7. a | 10. b | 13. c |
| 2. c | 5. c | 8. b | 11. d | 14. d |
| 3. a | 6. d | 9. c | 12. a | 15. b |

Short questions — model answers

1. AI is human-made ("artificial") technology that can perform tasks requiring thinking ("intelligence"); it uses data to make decisions. Examples: face recognition to unlock a phone; YouTube suggesting videos; chatbots answering questions.
2. IoT is a system in which physical devices are connected to the internet and can collect and share data to perform tasks automatically. Example: a smartwatch collects heart-rate data and sends it to a mobile phone for display.
3. A sensor collects information from the physical world (input; the "eyes and ears"), e.g. temperature sensor, motion sensor. An actuator performs a physical action in response to a command, converting digital signals into movement (output), e.g. electric motor, automatic door opener.
4. Cloud computing stores IoT data and provides processing services over the internet instead of local hard drives, so it can handle the massive amounts of data produced by many devices. Examples: Google Drive, iCloud, online LMS.
5. Collection, storage, cleaning, analysis, visualization. Cleaning removes duplicated records, incorrect values (typos), missing files/records, and improper formats.
6. Smart cities: sensors adjust traffic signals to reduce congestion; smart bins send alerts when full (also street lighting, parking). Healthcare: smartwatches measure heart rate and vitals; remote monitoring devices send patient data to doctors (also emergency alerts).
7. Benefits: students learn at their own speed; AI identifies students who need extra help (also supports students with disabilities). Challenges: limited internet access in rural areas; lack of computers/devices (also teacher training, data privacy).
8. Primary sources give original, firsthand information from people, events or official records (e.g. interviews, NADRA records). Secondary sources contain information already collected and explained by someone else, summarising or analysing primary data (e.g. textbooks, news reports).
9. When the information is outdated, lacks proper references/citations, or has not been verified by an expert.
10. (a) Communication: + emails and video calls give instant, global communication at almost no cost; – less in-person interaction causes social isolation. (b) Business: + online shopping, digital banking and automation improve speed and efficiency; – manual jobs are lost to machines and banks face cyber-attacks.

Long questions — model answers

1. **Definition:** IoT is a system in which physical devices are connected to the internet and collect and share data to perform tasks automatically. **Components:** (1) Sensors — collect information from the physical world, e.g. temperature, motion, fingerprint, smoke, soil moisture sensors. (2) Actuators — perform

physical actions in response to commands, converting digital signals to movement, e.g. motor, switch, door opener, water valve, fan controller. (3) Connectivity — lets devices send/receive data, e.g. Wi-Fi, Bluetooth, ZigBee, 4G/5G. (4) Data processing — changes raw data into useful information through collecting, organising, analysing and deciding, e.g. checking if a room is too hot to switch on the AC. (5) Cloud computing — stores data and provides services over the internet, handles massive data, e.g. Google Drive, iCloud, LMS. (6) User interface — lets users see information and give commands, e.g. mobile apps, touch screens, voice commands. **Diagram:** Sensor (collects) → Wi-Fi → Cloud (processes) → Actuator (acts) → User App (UI).

2. **Table:** IoT — network of physical devices connected to the internet; purpose: data collection and automation; works with hardware; output: raw data from environment. AI — technology enabling machines to think and decide; purpose: intelligent decisions and predictions; works with smart software; output: smart decisions or actions. Data Analytics — process of examining data to find patterns; purpose: extracting meaningful insights; works with data (numbers, records); output: reports, graphs, insights. **Working together:** IoT is the eyes that gather data, Analytics sorts it, AI is the brain that decides. Example: a temperature sensor reads 36 °C (IoT) → recent readings are organised showing the room is heating up (Analytics) → AI decides the room is too hot (AI) → a switch turns on the AC (action).
3. (1) **Collection:** gather marks of all Maths tests, attendance and survey responses from students. (2) **Storage:** keep them safely in a spreadsheet or database, e.g. on the cloud. (3) **Cleaning:** remove duplicate records, correct typing errors (e.g. 120 out of 100), fill or mark missing records, and make formats (dates, roll numbers) consistent. (4) **Analysis:** study the clean data to find patterns — e.g. which chapters most students got wrong, whether low attendance links to failing. (5) **Visualization:** present results in bar charts or a pie chart of pass vs fail so the principal sees the problem at a glance and plans extra classes for weak chapters.
4. **Primary:** original, firsthand, highly authentic (interviews, photographs, diaries, NADRA records). **Secondary:** collected and explained by someone else, summarises/analyses primary data (textbooks, news reports, research papers, Bureau of Statistics reports). **Tertiary:** highly summarised and organised from primary and secondary sources for quick reference (encyclopedias, Wikipedia, dictionaries, handbooks). **Classification:** (i) secondary, (ii) primary, (iii) tertiary, (iv) primary, (v) secondary, (vi) tertiary. **Unreliability:** primary — incomplete, recorded incorrectly, or deliberately false; secondary — outdated, no references/citations, not verified by an expert; tertiary — unknown/unverified website, or oversimplified and missing key facts. Rule: do not trust an article without author, date or references.
5. **Definition:** devices, software and systems that help people with disabilities or functional limitations participate equally in school, work and society. **Categories:** (1) Mobility aids — help people move; smart wheelchairs, stair lifts. (2) Sensory aids — support vision and hearing; hearing aids, screen readers (VoiceOver), Braille displays. (3) Communication aids — help people with speech difficulties express themselves; speech-generating devices, text-to-speech apps. (4) Cognitive/learning aids — help with memory, reading, understanding; audiobooks, spell-checkers, reminder apps. **Importance:** equal access to education (screen readers let blind students read digital textbooks), independence in daily life (mobility aids reduce need for constant help), social inclusion (people work, study and socialise confidently). **Careers:** any two of assistive software developer (speech-to-text tools), biomedical engineer (artificial limbs, hearing aids), UX/UI accessibility expert (websites usable by visually impaired users).
6. **Introduction:** computing makes life faster and more connected but brings privacy risks and addiction. **Education:** + online classes, e-books and learning apps make study flexible and accessible; – excessive screen time reduces attention span and harms physical health. **Healthcare:** + digital patient records and telemedicine let doctors treat remote patients accurately (e.g. rural Sindh); – patient data privacy is at risk if hospital servers are hacked. **Agriculture:** + smart irrigation and weather forecasting greatly increase yields; – poor rural internet means many farmers cannot use these tools. **Responsible use:** protect personal information and use strong passwords; do not share fake news or unverified forwards; balance screen time with sports and physical activity. **Conclusion:** benefits are large if technology is used wisely.

Self-assessment: I can...

- ☐ define AI and explain why it is called "artificial intelligence".
- ☐ explain how IoT, Data Analytics and AI work together (eyes → sort → brain → action).
- ☐ define IoT and name its six components with examples.
- ☐ tell the difference between a sensor and an actuator.
- ☐ list the five components of Data Analytics in order and describe data cleaning.
- ☐ compare IoT, AI and Data Analytics in a table.
- ☐ describe IoT uses in smart cities, industry, healthcare, education, homes and farms.
- ☐ state uses, benefits and challenges of AI in education in Pakistan.
- ☐ classify any source as primary, secondary or tertiary and judge its reliability.
- ☐ discuss positive and negative impacts of computing in five fields.
- ☐ explain the four categories of assistive technologies and their importance.
- ☐ name careers linked to IoT, AI and assistive technology and the skills they need.